

Full Technical Appendix

Phases 0 & 1 — every document, every line of code, every concept, and the system as built

24 July 2026 · 58 files reproduced in full · Standalone reference

Contents

1. Part A — Concepts implemented in Phases 0 & 1

2. A. All concepts, condensed

3. Part B — Every document and every line of code

4. B1. Planning & reference documents

5. B2. Database (SQL)

6. B3. Backend — Python agent: core

7. B4. Backend — Python agent: database layer

8. B5. Backend — Python agent: stub modules

9. B6. Backend — package init files

10. B7. Frontend — dashboard configuration

11. B8. Frontend — dashboard application code

12. B9. Tooling

13. Part C — Visual workflow

14. C. How everything connects

A. All concepts implemented in Phases 0 & 1, condensed

Each concept below was applied for real somewhere in the files that follow in Part B — this is the "why," collected in one place before the "what."

Phase 0 concepts

Version control (Git)

Every change tracked as a chain of commits; a staging area lets a commit be built deliberately; a local repo and the GitHub remote are separate, connected only by an explicit `origin` pointer.

Secret hygiene before first commit

`.gitignore` was written before anything else specifically because this repository is public — a committed secret is recoverable from history forever, even after later deletion.

Python virtual environments

An isolated, project-scoped copy of the interpreter and its packages, so this project's dependencies can never collide with any other Python project on the machine.

Dependency resolution

Exact version pins (`==`) left no room for two packages' conflicting sub-requirements to be reconciled; minimum pins (`>=`) gave the solver room to find a version that satisfies everyone.

Python packages and modules

`__init__.py` marks a folder as importable; dot-notation imports (`agent.db.repositories`) walk that package structure.

Centralised configuration

Exactly one file (`config.py`) reads environment variables; every other module asks it for a named value, never the OS directly.

Node.js / npm / package.json

The JavaScript equivalent of the Python `venv` + `requirements.txt` pairing — an isolated runtime and a declared, reproducible dependency list.

Build tools (Vite)

Transforms JSX/ES-module source into plain JS/CSS/HTML a browser can run; instant reload in development, a fixed optimised bundle for production.

Component-based UI (React) & SPA

The interface is built from small functions returning markup (JSX); one HTML page loads once, and JavaScript swaps content in afterward without full reloads.

Utility-first CSS (Tailwind v4)

Styling composed from small utility classes in markup; v4's plugin-based integration removed the separate config files earlier versions required.

Progressive Web Apps

A manifest, icon set, and iOS-specific meta tags make a website installable and launchable full-screen from a phone's home screen.

Binary file formats (PNG)

Icons were generated with a hand-written PNG encoder — signature bytes, chunked structure (IHDR/IDAT/IEND), CRC32 checksums, zlib-compressed pixel data — with zero external image libraries.

Backend-as-a-Service

Supabase supplies a managed Postgres database, an auto-generated REST API over it, authentication, and a realtime layer, replacing a hand-built backend server.

JWTs and least-privilege API keys

Both Supabase keys are signed JSON Web Tokens declaring a role; the low-privilege anon key is safe to expose publicly only because Row Level Security constrains what it can do, while the powerful service-role key is confined to the trusted server-side agent only.

Phase 1 concepts

Relational database structure

Nine tables, each one entity, related to one another rather than duplicating data.

SQL DDL vs DML

`CREATE TABLE` statements define structure (DDL); the day-to-day reads and writes the agent performs are DML, generated for us by the client library.

UUID primary keys

128-bit identifiers generated independently by any process with no central coordination and negligible collision risk — chosen over auto-incrementing integers because both the agent and dashboard can create rows concurrently.

Foreign keys & referential integrity

`references` constraints, with explicit `on delete cascade` / `on delete set null` behaviour, so the database itself refuses to let a relationship point at nothing.

JSONB columns

Variable-shaped data (a lead's weak-points list) stored as a queryable JSON document inside one column, rather than forcing a rigid extra table for a simple list.

Indexes

Separate lookup structures built on the columns queried most often (status, temperature, platform), trading a little storage/write cost for much faster reads.

Row Level Security

Enabling RLS flips a table's default from open to closed; explicit policies then grant back exactly the access needed — here, full access for any `authenticated` user, none for anyone else. The service-role key bypasses this layer entirely by design.

Repository design pattern

Every database read/write funnelled through one file of small, purpose-named functions, so a schema change only ever requires editing one place.

REST over Postgres (PostgREST)

The Python client never writes raw SQL during normal operation — it builds HTTP requests against an API Supabase auto-generates directly from the schema.

Smoke testing & round-trip verification

A fast, broad check (connection, tables, seed data) plus a genuine round-trip: insert, read back, confirm an exact match, then delete — leaving no trace.

DNS, IPv4 vs IPv6, connection pooling

A hostname must resolve via DNS before it's reachable; Supabase's direct-connection host is IPv6-only, unreachable on an IPv4-only network, which is exactly what was hit — solved by switching to the IPv4-reachable session-pooler endpoint instead.

Least privilege

Every component was given only the access it needs — anon+RLS for the public dashboard, service-role for the trusted agent, and a one-time DB password used only transiently and then discarded rather than stored anywhere permanent.

B1. Planning & Reference Documents

00_MASTER_PROMPT.md

78 lines · markdown

The instruction set the whole build follows

```
# MASTER PROMPT – Nexaris AI Outreach Agent

> Paste this entire file into Claude Code as your first message.
> Attach the other 3 files (`01_PHASED_BUILD_PLAN.md`, `02_SUPABASE_SCHEMA.sql`, `03_PROJECT_STRUCTURE.md`) alongside it.

---

## Who you are

You are the lead engineer building a production AI outreach agent for Nexaris, an AI/software agency. You are working with Hussein, the owner. Hussein is a CS student and a capable developer, but he wants to understand every part of what you build – he is not looking for a black box.

## How you must behave (read carefully – this is the most important part)

1. Work in phases. Follow the phased build plan in `01_PHASED_BUILD_PLAN.md` exactly. Do not jump ahead. Do not build Phase 3 before Phase 2 is done and confirmed.

2. Explain every single step. Before writing code for a phase, explain in plain language: what you are about to build, why, and how it connects to the rest. After writing it, explain what the code does, file by file. Hussein explicitly asked to be informed of every detail – the same way a previous project (a marriage platform) was built with him.

3. Confirm before moving on. At the end of every phase, stop. Summarize what was built, show Hussein how to test it, and ask him to confirm it works before you start the next phase. Never chain multiple phases without a checkpoint.

4. Never skip details from the spec. The complete specification is in the attached documents and summarized below. If something is ambiguous, ask Hussein rather than guessing. If you notice a requirement you can't fully implement yet, flag it clearly rather than silently dropping it.

5. Keep code clean and commented. Every non-obvious function gets a short comment explaining what it does. Hussein should be able to read the code and follow it.

6. Track progress. Maintain a `PROGRESS.md` file in the repo. After each phase, update it with what's done, what's next, and any decisions made. This lets Hussein (or a fresh session) pick up without losing context.

---

## What you are building – one paragraph

A fully automated AI outreach agent that runs daily on a schedule, uses 3 accounts (each on its own proxy) to discover business leads on LinkedIn and Instagram, deeply analyzes each business, detects founders, finds public WhatsApp numbers, scores each lead 1-10 with reasoning, and generates a personalized human-style outreach message using the Claude API. Messages wait for Mahmoud's daily approval, then send automatically on LinkedIn and WhatsApp – while Instagram messages are surfaced in the dashboard for a human to send manually (Instagram bans automated cold outreach). Everything lands in a CRM with a pipeline, and a responsive dashboard (works on laptop and iPhone) shows live leads, analytics, account health, and every editable setting.

---

## Tech stack (already decided – do not change without asking)

- Frontend / dashboard: React + Tailwind CSS (responsive PWA – must work on laptop and iPhone, installable via "Add to Home Screen")
- Backend / agent: Python
- Browser automation: Playwright (for LinkedIn/Instagram discovery + LinkedIn sending)
- Agent brain: Claude API
  - Haiku model → analysis, scoring, qualification, founder detection (cost-efficient)
  - Sonnet model → message generation (higher writing quality, better reply rates)
```

- **Database:** Supabase (Postgres + Auth + Realtime) – **free tier**, only 2 users
- **Dashboard hosting:** Vercel (free tier)
- **Agent hosting:** always-on server for scheduled runs – Oracle Cloud free tier or a ~\$5/month VPS (Hetzner). Recommend VPS for reliability.
- **Proxies:** residential sticky proxies, **one dedicated IP per account** – config slot per account, can start empty and be filled later

The 10 core rules of this system (never violate)

1. **Instagram is find-only.** The agent discovers, analyzes, scores, and drafts messages for Instagram leads but **never sends** on Instagram. Those messages go to a dashboard section for manual human sending. (Instagram's API bans automated cold outreach.)
2. **LinkedIn is fully automated** – find and send.
3. **WhatsApp sends automatically** – but only when a public WhatsApp number is found on a lead's Instagram bio, LinkedIn profile, or website.
4. **Nothing sends without Mahmoud's daily approval.** This is a hard gate with no bypass.
5. **Messages are AI-generated** per lead, personalized from the business name + details + weak points, in a human tone.
6. **Each of the 3 accounts uses its own dedicated sticky proxy IP** – sessions never mix IPs.
7. **Founder detection:** scan bios for "founder of X", "co-founder", "CEO & founder", etc. If found, flag on the dashboard for manual founder outreach.
8. **Max 2 contacts per lead.** Follow-up is per-lead (Hussein sets on/off + timing). Re-contact gap is Hussein's choice.
9. **Warnings show type + reason.** Redistribution of a paused account's quota is Hussein's manual decision, never automatic.
10. **Everything is editable from the dashboard** – niche, location, language, per-account run times, per-account daily limits, message style, style duration, follow-up, proxy credentials, WhatsApp recipient numbers.

Full feature reference

The complete, verified specification is in the attached file `Nexaris\_Agent\_COMPLETE\_SPEC\_FINAL.pdf` (Hussein has it). The build plan below references it. If you ever need a detail not in the build plan, ask Hussein to quote the relevant section of that spec.

Start here

1. Read all attached files fully.
2. Confirm back to Hussein: your understanding of the project in your own words (5–8 sentences), the tech stack, and the phase list.
3. Ask any clarifying questions you have BEFORE writing any code.
4. Then begin **Phase 0** from `01\_PHASED\_BUILD\_PLAN.md`.

Do not write any code until you've done steps 1–3 above.

The 11 phases, in order

```
# PHASED BUILD PLAN – Nexaris AI Outreach Agent

Build in this exact order. **Stop at the end of every phase, summarize, show how to test, and wait for Hussein's confirmation before continuing.**

Update `PROGRESS.md` after each phase.

---

## PHASE 0 – Project setup & foundations

**Goal:** a clean, running skeleton for both the agent and the dashboard.

- Set up the repo using the structure in `03_PROJECT_STRUCTURE.md`
- Initialize the Python backend/agent project (with a virtual environment, `requirements.txt`)
- Initialize the React + Tailwind dashboard (Vite)
- Set up the Supabase project connection (env vars, client setup in both backend and frontend)
- Create a `.env.example` listing every environment variable the project will need (Claude API key, Supabase URL/key, proxy credentials per account, WhatsApp API credentials, etc.)
- Create `PROGRESS.md`
- Confirm both apps run (dashboard shows a blank page, agent runs a "hello" script)

**Checkpoint:** Hussein can run the dashboard locally and run the agent script. Nothing does real work yet.

---

## PHASE 1 – Database schema

**Goal:** the full database exists in Supabase.

- Run `02_SUPABASE_SCHEMA.sql` in Supabase (walk Hussein through doing this)
- Verify every table exists: `leads`, `messages`, `pipeline_history`, `client_history`, `accounts`, `settings`, `notifications_log`, `follow_ups`
- Set up Row Level Security appropriately (2 users only)
- Create the backend data-access layer (functions to read/write each table)
- Seed the `settings` table with sensible defaults (niche empty, 20 IG / 30 LinkedIn per account, default run times, message style = "discovery", style duration = 7 days, etc.)

**Checkpoint:** Hussein can see all tables in Supabase and the backend can read/write a test record.

---

## PHASE 2 – Agent core: account pool & browser sessions

**Goal:** the agent can open isolated browser sessions per account, each on its own proxy.

- Build the **account pool manager**: loads the 3 accounts, each with its own run time, daily limit, and **proxy config slot** (can be empty for now)
- Build Playwright session management: one isolated browser context per account, routed through that account's proxy IP (if set) – sessions must NEVER share context or IP
- Build the **account health check**: detect login challenges, CAPTCHAs, restrictions; when detected, record warning **type + reason**, pause the account, and log it
- Build the **scheduler**: each account runs at its own configured time; agent records run start and **finish time**
- Build the **redistribution decision hook**: when an account is paused, do NOT auto-redistribute – surface the decision to Hussein (a flag in the DB the dashboard will read)

**Checkpoint:** Hussein can trigger a run; the agent opens sessions per account (no proxy yet is fine), respects run times, and logs a warning correctly if a session fails. Test at LOW volume.

---

## PHASE 3 – Discovery: LinkedIn & Instagram

**Goal:** the agent finds and reads real profiles.
```

- **LinkedIn discovery:** search by niche/industry/company size/location/activity; read company name, description, headcount, website, recent posts, public contact info; target 30/account (from settings)
- **Instagram discovery:** search hashtags/explore/location tags; read bio, followers, posts, engagement; target 20/account (from settings)
- **Qualification logic:** decide if each profile is a real business worth pursuing; skip personal/inactive/irrelevant accounts
- Make discovery **agentic** – if a search returns weak results, the agent adapts its search terms rather than collecting junk
- Store raw discovered data in `leads` with status "discovered"

Checkpoint: Hussein runs the agent and sees real qualified leads appear in the database from both platforms. Test at low volume with a few profiles.

PHASE 4 – Claude analysis pipeline

Goal: every lead is deeply analyzed, scored, and enriched by Claude.

- **Deep analysis (Haiku):** company size, revenue tier, industry, website analysis, ads activity, social presence, full weak points list, AI opportunities
- **Founder detection (Haiku):** scan bio for founder phrases; if found, store founder name + set the founder flag
- **WhatsApp number detection:** check bio/description/website for a public WhatsApp number; store if found
- **Lead scoring (Haiku):** score 1-10 with written reasoning; label Hot/Warm/Cold
- **Save to `client\_history`** permanently (full record, whether or not it's later contacted)
- Use **prompt caching** on the repeated instruction template to cut cost
- Store everything on the `leads` record

Checkpoint: Hussein sees leads in the DB now fully enriched – score, reasoning, weak points, founder flag, WhatsApp flag. Verify the scoring reasoning makes sense on real examples.

PHASE 5 – Message generation

Goal: every qualified lead gets a personalized, human-sounding message.

- **Message generation (Sonnet):** personalized greeting (business/team/founder name), references specific weak points/opportunities, human tone
- Respect the **message style** setting (Direct = names weak points; Discovery = softer/curiosity) and **style duration** (rotate after N days)
- Use the personalization-line style from the spec as a guide (e.g. "I noticed you're spending on Meta ads, but your follow-up is manual and your booking takes 12h")
- Adapt tone per platform (LinkedIn more formal, WhatsApp more direct)
- Store the generated message on the lead, status "awaiting_approval"

Checkpoint: Hussein reviews generated messages for real leads and confirms they read as human and personalized.

PHASE 6 – Approval workflow

Goal: Mahmoud approves before anything sends.

- Build the message queue: all generated messages sit in "awaiting_approval"
- Build the backend approve/edit/hold logic
- Nothing can move to "sending" without approval
- Build the approval reminder trigger (feeds Phase 8 notifications)

Checkpoint: Hussein can approve/edit/hold a message via a test call; unapproved messages are correctly blocked from sending.

PHASE 7 – Sending & routing

Goal: approved messages go out on the right channels.

- **LinkedIn:** auto-send via the account's Playwright session; log timestamp/account/platform; move lead to "Contacted"
- **WhatsApp:** auto-send when a public number was found (integrate WhatsApp Business API – Twilio/360dialog, or the chosen method); log and move to "Contacted"
- **Instagram:** do NOT auto-send – mark the message as "manual_send_pending" so the dashboard's Instagram section shows it; when a human marks it Sent, move to "Contacted"
- Respect per-account daily limits and the warm-up ramp

Checkpoint: A test approved LinkedIn message sends; a WhatsApp test sends; an Instagram message correctly appears in the manual-send queue instead of sending.

PHASE 8 – CRM, pipeline, follow-up, notifications

Goal: the full lifecycle and all alerts work.

- **Pipeline:** New → Contacted → Replied → Interested → Meeting Booked → Deal Closed → Lost; auto-move on reply detection; manual moves supported; timestamp every move
- **Follow-up + re-engagement:** per-lead on/off + timing; re-engagement for cold-after-reply; max 2 contacts; re-contact gap set per lead; pause on reply
- **WhatsApp notifications** (all 5 types to both numbers): hot lead alert, founder found, account warning, approval reminder (to Mahmoud), daily run summary
- Reply detection: check for replies on LinkedIn/WhatsApp and update pipeline

Checkpoint: Hussein sees leads move through stages, receives test WhatsApp alerts, and follow-up scheduling works.

PHASE 9 – Dashboard frontend

Goal: the full responsive dashboard, working on laptop and iPhone.

Build these sections (see spec §7 for full detail):

- **Live Feed** – today's lead cards (all details incl. weak points, founder flag, copyable message)
- **Instagram Manual Send section** – ready messages + "Mark as Sent"
- **Mahmoud's Approval section** – edit/approve per message + Approve All + count badge (mobile-friendly)
- **CRM Pipeline board** – kanban, drag-and-drop
- **Lead Detail page** – full record + editable follow-up
- **Client History** – permanent searchable database
- **Analytics/KPIs** – all metrics from spec §7.7 with date/platform/account filters
- **Account Health monitor** – status + warning type/reason + redistribution toggle
- **Run Status panel** – running/idle, current task, start/finish time, duration
- **Settings panel** – every editable control from spec §8
- Make it a **PWA** (installable, works full-screen on iPhone)

Checkpoint: Hussein opens the dashboard on his laptop AND adds it to his iPhone home screen; every section shows real data.

PHASE 10 – Deploy & warm-up

Goal: live and safe.

- Deploy dashboard to Vercel
- Deploy agent to the always-on server (Oracle free tier or VPS)
- Set up the daily scheduler on the server
- **Test end-to-end** at LOW volume with NO proxy first
- Add proxy credentials per account in settings (3 dedicated sticky residential IPs)
- Begin the **warm-up ramp**: start 5-10/day per account, increase over 2-4 weeks to the 20 IG / 30 LinkedIn target
- Vary timing slightly between accounts

Checkpoint: The system runs a full real daily cycle end-to-end, safely, at warm-up volume.

After Phase 10

Everything in the spec's "Out of Scope" section (meetings automation, proposals, call summaries, support mode, campaigns) is a potential **Phase 2 project** – do not build these now.

The intended folder layout

```
# PROJECT STRUCTURE – Nexaris AI Outreach Agent
```

This is the folder layout Claude Code should set up in Phase 0. Two main parts: the **agent** (Python) and the **dashboard** (React). They share one Supabase database.

```
...
nexaris-outreach-agent/
├── PROGRESS.md           # updated after every phase – what's done, what's next
├── README.md            # how to run everything
├── .env.example         # every env var needed (no real secrets)
├── agent/               # — THE PYTHON AGENT —
│   ├── requirements.txt
│   ├── .env             # real secrets (gitignored)
│   ├── main.py          # entry point – the daily run loop
│   ├── scheduler.py     # runs each account at its own time
│   ├── core/
│   │   ├── account_pool.py # loads 3 accounts, manages rotation
│   │   ├── session.py     # Playwright browser context per account + proxy
│   │   ├── health.py      # detects warnings, records type + reason
│   │   └── warmup.py      # per-account warm-up ramp logic
│   ├── discovery/
│   │   ├── linkedin.py   # LinkedIn search + profile reading
│   │   ├── instagram.py  # Instagram search + profile reading
│   │   └── qualify.py    # is this a real business worth pursuing?
│   ├── analysis/
│   │   ├── analyze.py    # deep company analysis (Claude Haiku)
│   │   ├── founder.py    # founder detection from bio
│   │   ├── whatsapp_detect.py # find public WhatsApp number
│   │   ├── score.py      # lead scoring 1-10 + reasoning (Haiku)
│   │   └── prompts.py    # all Claude prompt templates (with caching)
│   ├── messaging/
│   │   ├── generate.py   # message generation (Claude Sonnet)
│   │   └── style.py      # message style + duration rotation logic
│   ├── sending/
│   │   ├── linkedin_send.py # auto-send on LinkedIn
│   │   ├── whatsapp_send.py # auto-send on WhatsApp (when number found)
│   │   └── instagram_queue.py # push to dashboard manual-send queue (NO auto-send)
│   ├── crm/
│   │   ├── pipeline.py   # stage movement + history
│   │   ├── followup.py   # follow-up + re-engagement scheduling (max 2 contacts)
│   │   └── reply_detection.py # detect replies, move pipeline
│   ├── notifications/
│   │   └── whatsapp_notify.py # all 5 alert types to both numbers
│   └── db/
│       ├── client.py     # Supabase connection
│       └── repositories.py # read/write functions per table
├── dashboard/          # — THE REACT DASHBOARD (PWA) —
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── .env            # frontend env (Supabase public keys)
│   ├── public/
│   └── manifest.json    # PWA manifest (installable on iPhone)
```

```

├── icons/                # app icons for home screen
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── lib/
│   │   └── supabase.js    # Supabase client
│   ├── components/       # shared UI pieces
│   │   ├── LeadCard.jsx
│   │   ├── KpiCard.jsx
│   │   ├── TemperatureBadge.jsx
│   │   └── ...
│   └── pages/            # one file per dashboard section (spec §7)
│       ├── LiveFeed.jsx   # today's leads
│       ├── InstagramManualSend.jsx # IG messages to send by hand
│       ├── ApprovalQueue.jsx # Mahmoud approves messages
│       ├── PipelineBoard.jsx # kanban
│       ├── LeadDetail.jsx  # full lead record
│       ├── ClientHistory.jsx # permanent database
│       ├── Analytics.jsx   # all KPIs + charts
│       ├── AccountHealth.jsx # 3 accounts, warnings, redistribution
│       ├── RunStatus.jsx   # live run tracking + finish time
│       └── Settings.jsx    # every editable control (spec §8)
└── database/
    └── schema.sql         # copy of 02_SUPABASE_SCHEMA.sql for reference
...

```

Notes for Claude Code

- **Agent and dashboard are separate apps** sharing one Supabase DB. The agent writes data; the dashboard reads it (and writes settings/approvals/manual actions back).
- **Realtime:** use Supabase Realtime so the dashboard updates live as the agent works (live feed, run status, analytics).
- **The PWA manifest + icons** are what make it installable on Mahmoud's iPhone – don't skip these in Phase 9.
- **Secrets** (Claude API key, proxy credentials, WhatsApp API) live in `agent/.env`, never committed. `.env.example` documents them.
- Keep each file focused – one responsibility. Hussein wants to be able to open any file and understand it.

Original usage instructions for the planning package

HOW TO USE THIS PACKAGE – for Hussein

You have 4 files. Here's exactly what to do with them in Claude Code.

The files

1. ****00_MASTER_PROMPT.md**** – the main instruction. This tells Claude Code who it is, what it's building, the rules, and how to behave (explain everything, work in phases, confirm before moving on).
2. ****01_PHASED_BUILD_PLAN.md**** – the 11 phases (Phase 0 → Phase 10), in order.
3. ****02_SUPABASE_SCHEMA.sql**** – the full database, ready to run in Supabase.
4. ****03_PROJECT_STRUCTURE.md**** – the exact folder layout.

Plus keep ****Nexaris_Agent_COMPLETE_SPEC_FINAL.pdf**** handy – it's the full reference if Claude Code needs a detail.

Steps

****1. Set the model to Opus**** in Claude Code (`/model` → Opus). Use Opus for Phases 0–4 (the complex foundation). You can switch to Sonnet later for routine dashboard work.

****2. Start a new Claude Code session**** in an empty folder where you want the project built.

****3. Paste the entire contents of `00\_MASTER\_PROMPT.md`**** as your first message, and attach the other 3 files (and the spec PDF) to the same message.

****4. Let Claude Code respond.**** It will:

- Read everything
- Tell you its understanding of the project in its own words
- Ask you any clarifying questions
- Wait for your go-ahead

****5. Answer its questions****, then tell it to begin Phase 0.

****6. After each phase****, Claude Code will stop, summarize what it built, show you how to test it, and ask you to confirm before continuing. Test it, confirm, and it moves to the next phase.

Things you'll need to have ready

Before Phase 4 (Claude analysis) you need:

- ****A Claude API key**** (from console.anthropic.com)

Before Phase 1 (database) you need:

- ****A Supabase project**** (free – supabase.com) with its URL and keys

Before Phase 7 (WhatsApp sending) you need:

- A ****WhatsApp Business API**** account (Twilio or 360dialog) OR decide on the informal method

Before Phase 10 (deploy + real use) you need:

- ****3 dedicated sticky residential proxy IPs**** (DataImpulse / IPRoyal / Evomi) – one per account
- ****An always-on server**** (Oracle Cloud free tier or a ~\$5/month Hetzner VPS)
- ****A Vercel account**** (free) for the dashboard

You do NOT need the proxies or server until Phase 10 – you can build and test everything before then locally with no proxy at low volume.

Important reminders (Claude Code has these too, but so you know)

- ****Instagram never auto-sends**** – the agent prepares those messages, a human sends them from the dashboard.
- ****Nothing sends without Mahmoud's daily approval.****
- ****Each account gets its own proxy IP**** – never shared.
- ****Warm up slowly**** – start at 5–10/day per account, ramp up over 2–4 weeks. Do not start at full volume with the shared-origin accounts.

If a session runs out of context

Claude Code keeps a `PROGRESS.md` file in the repo. If you start a fresh session, point it at `PROGRESS.md` and the master prompt, and it can continue where it left off.

Project front door

Nexaris AI Outreach Agent

An automated outreach agent that discovers business leads on LinkedIn and Instagram, analyses each business with Claude, scores it 1-10 with written reasoning, and writes a personalised message – which sends automatically on LinkedIn and WhatsApp only after daily human approval. Instagram messages are prepared but always sent by hand. Everything lands in a CRM with a pipeline, shown in a responsive dashboard that works on laptop and iPhone.

Status: Phase 0 of 11 complete. See PROGRESS.md.

The three channel rules

Channel	Discovery	Sending
LinkedIn	Automated	Automated (after approval)
WhatsApp	–	Automated (after approval), only when a public number is found
Instagram	Automated	Never automated – prepared for a human to send manually

Instagram's terms ban automated cold outreach, so the agent finds, analyses, scores, and writes the message, then stops. Nothing sends on any channel without Mahmoud's daily approval – that gate has no bypass.

Running it

Agent (Python)

```

`bash
# One-time setup
python -m venv agent/venv
agent/venv/Scripts/python.exe -m pip install -r agent/requirements.txt
agent/venv/Scripts/python.exe -m playwright install chromium # from Phase 2
cp .env.example agent/.env # then fill it in

# Run
agent/venv/Scripts/python.exe -m agent.main
`

```

On macOS or Linux use `agent/venv/bin/python` instead.

Dashboard (React PWA)

```

`bash
cd dashboard
npm install
cp ../.env.example .env # keep only the VITE_ lines, then fill them in
npm run dev
`

```

Opens on `http://localhost:5173`. The dev server binds to your local network too, so you can open it on a phone using your laptop's IP to test the mobile layout.

Layout

agent/	Python agent – discovery, analysis, messaging, sending, CRM
core/	Account pool, browser sessions, health checks, warm-up
discovery/	LinkedIn + Instagram search and qualification

```
analysis/      Claude analysis, founder detection, scoring
messaging/    Message generation and style rotation
sending/      Channel routing
crm/          Pipeline, follow-up, reply detection
db/           Supabase access
dashboard/    React + Tailwind PWA
database/     Reference copy of the Supabase schema
...
```

The agent and dashboard are separate apps sharing one Supabase database. The agent writes data; the dashboard reads it and writes back settings, approvals, and manual actions. Supabase Realtime keeps the dashboard live as the agent works.

Reference documents

```
| File | What's in it |
|-----|-----|
| [00_MASTER_PROMPT.md](00_MASTER_PROMPT.md) | How the build is run, the 10 core rules |
| [01_PHASED_BUILD_PLAN.md](01_PHASED_BUILD_PLAN.md) | The 11 phases in order |
| [02_SUPABASE_SCHEMA.sql](02_SUPABASE_SCHEMA.sql) | Full database schema |
| [03_PROJECT_STRUCTURE.md](03_PROJECT_STRUCTURE.md) | Intended folder layout |
| [REQUIREMENTS_COVERAGE.md](REQUIREMENTS_COVERAGE.md) | Every requirement mapped to a phase |
| [PROGRESS.md](PROGRESS.md) | What's built, what's next, decisions made |
```

Stack

React + Tailwind (PWA on Vercel) · Python + Playwright (agent on an always-on server) · Claude API – Haiku for analysis, Sonnet for messages · Supabase for Postgres, Auth, and Realtime · one dedicated sticky residential proxy IP per account.

Phase tracker and decisions log

```
# PROGRESS — Nexaris AI Outreach Agent
```

Updated after every phase. If a session runs out of context, point the next one at this file plus `00\_MASTER\_PROMPT.md` and it can pick up without losing anything.

****Current state:**** Phase 1 complete and verified live against the real Supabase project.

****Next:**** Phase 2 — agent core: account pool & browser sessions.

```
---
```

Phase status

Phase	Name	Status
0	Project setup & foundations	✅ Complete
1	Database schema	✅ Complete — verified live in Supabase
2	Agent core: account pool & sessions	❌ Not started
3	Discovery: LinkedIn & Instagram	❌ Not started
4	Claude analysis pipeline	❌ Not started
5	Message generation	❌ Not started
6	Approval workflow	❌ Not started
7	Sending & routing	❌ Not started
8	CRM, pipeline, follow-up, notifications	❌ Not started
9	Dashboard frontend	❌ Not started
10	Deploy & warm-up	❌ Not started

```
---
```

Phase 0 — Project setup & foundations

****Built:****

- Git repository initialised on `main`, `.gitignore` covering `.env`, `venv/`, `node\_modules/`, and Playwright browser profiles
- Folder structure exactly per `03\_PROJECT\_STRUCTURE.md`
- ****Python agent**** (`agent/`)
 - `venv/` with all dependencies installed
 - `requirements.txt` — supabase, anthropic, playwright, APScheduler, dotenv, httpx, beautifulsoup4, pytz
 - `config.py` — single place all secrets and settings are read from
 - `db/client.py` — lazy Supabase client using the service role key
 - `main.py` — runs a self-check reporting what is and isn't configured
 - 23 stub modules, each documenting what it will hold and which phase builds it
- ****Dashboard**** (`dashboard/`)
 - Vite + React 18 + Tailwind v4
 - `src/lib/supabase.js` — client using the anon key, degrades gracefully when unset
 - `src/App.jsx` — Phase 0 status page listing the 10 sections coming in Phase 9
 - PWA manifest + generated icons (180/192/512 + SVG favicon), iOS meta tags, safe-area padding for iPhone full-screen
- `.env.example` documenting every variable, annotated with the phase that needs it
- `database/schema.sql` — copy of the Supabase schema for reference
- `REQUIREMENTS\_COVERAGE.md` — every spec requirement mapped to a phase, so nothing can be silently dropped
- ****Documentation**** — `docs/PHASE\_0\_EXPLAINED.pdf` (17 pages) explaining every file, decision, and problem hit, plus step-by-step next actions. Generated by `tools/make\_pdf.py`, which renders HTML to PDF using the Chromium that Playwright already installs. Every phase from here produces one of these.

****How to test:****

```
```bash
Agent
agent/venv/Scripts/python.exe -m agent.main
```

```
Dashboard
cd dashboard && npm run dev
```

Decisions made:

1. Built in place rather than inside a `nexaris-outreach-agent/` wrapper folder, so the planning documents sit alongside the code. Structure below that level is exactly as specified.
2. Tailwind v4 via the Vite plugin instead of v3. No `tailwind.config.js` or `postcss.config.js` is needed – theme customisation lives in `src/index.css` under `@theme`. This differs from `03_PROJECT_STRUCTURE.md`, which was written assuming v3.
3. Dependency pins loosened to `>=. Exact pins made the set unresolvable – supabase and anthropic each constrain `httpx` to different ranges.
4. runs` table treated as a 9th table to verify in Phase 1. It is created by the schema but missing from the build plan's verification list, and it is what feeds the Run Status panel and the finish time in the daily summary.
5. Icons are placeholders – a generated "N" mark. Replace with real Nexaris branding in Phase 9.

Not done / still open:

- Supabase project not created yet – blocks Phase 1
- Claude API key not set – blocks Phase 4
- Playwright browsers not downloaded yet (`playwright install chromium`) – Phase 2
- Open decisions Q1-Q7 in `REQUIREMENTS_COVERAGE.md`

---

## Phase 1 – Database schema

Built (code, no live DB yet):

- `database/policies.sql` – Row Level Security for all 9 tables. Model: RLS on, `authenticated` role gets full access, `anon` gets nothing. Fits the 2-user setup. The agent's service-role key bypasses RLS, so only the dashboard is governed.
- `agent/db/repositories.py` – the data-access layer. 25 functions across all 9 tables, one section per table. The only module that touches the database directly. Verified: imports clean.
- `agent/db/smoke_test.py` – Phase 1 checkpoint proof. Checks connection, all 9 tables, seed data (3 accounts + settings), and a self-cleaning write/read/delete. Verified: fails gracefully with a clear message when keys are absent.

Live verification (2026-07-24): Supabase project created (ref `xtesxgezongzpdgppobm`, Singapore). Both SQL files run successfully against the real database via a session-pooler connection (the direct/IPv6 connection failed to resolve on this network – documented in the Phase 1 PDF). `python -m agent.db.smoke_test` – ALL CHECKS PASSED: connection, all 9 tables, seed data (3 accounts + settings), and a clean insert/read/delete round-trip.

Phase 1 PDF: `docs/PHASE_1_EXPLAINED.pdf` – covers all 3 files, the migration process, the IPv6/pooler issue, and a security note on the temporary use of the DB password.

---

## Decisions log



Date	Decision
2026-07-22	Repo is public on GitHub as `AiOutreachAgent`. Hussein pushes himself – the agent never runs `git push`.
2026-07-22	Build order follows `01_PHASED_BUILD_PLAN.md` (11 phases) over spec §14 (10 steps) where they conflict. No requirement is dropped either way.


```

Every spec requirement mapped to a phase

REQUIREMENTS COVERAGE LEDGER – Nexaris AI Outreach Agent

Every requirement from `Nexaris\_Agent\_COMPLETE\_SPEC\_FINAL.pdf`, the master prompt, and the build plan is listed here with an ID, the phase that delivers it, and the file it lands in.

Purpose: Hussein asked that nothing from the spec be missed. A promise isn't checkable – this table is. At the end of every phase, the rows for that phase get marked `DONE` and the phase is not closed until all of its rows are marked.

Status legend: `TODO` = not started · `WIP` = in progress · `CODE` = code written, not yet verified live · `DONE` = built and tested by Hussein · `N/A` = deliberately out of scope

Phase 1 – DONE (2026-07-24). `database/policies.sql`, `agent/db/repositories.py`, and `agent/db/smoke\_test.py` are built and verified live against the real Supabase project (ref `xtesxgezongzpdgppobm`). All 9 tables exist with RLS enabled; seed data confirmed; smoke test passed end-to-end. Q6 resolved. Settings rows S1-S8, S14 have their storage in place; their dashboard UI is still Phase 9.

A. The 10 core rules (never violated – these are constraints, not features)

| ID | Rule | Enforced in | Phase | Status |
|------|--|-----------------------------------|-------|--------|
| ---- | ----- | ----- | ----- | ----- |
| R1 | Instagram is find-only – never auto-sends | `sending/instagram_queue.py` | 7 | TODO |
| R2 | LinkedIn is fully automated – find and send | `sending/linkedin_send.py` | 7 | TODO |
| R3 | WhatsApp auto-sends only when a public number was found | `sending/whatsapp_send.py` | 7 | TODO |
| R4 | Nothing sends without Mahmoud's approval – hard gate, no bypass | approval gate in `sending/` | 6 | TODO |
| R5 | Messages are AI-generated per lead, human tone | `messaging/generate.py` | 5 | TODO |
| R6 | Each account uses its own dedicated sticky proxy IP – never shared | `core/session.py` | 2 | TODO |
| R7 | Founder detection flags for manual outreach – never auto-contacts founder | `analysis/founder.py` | 4 | TODO |
| R8 | Max 2 contacts per lead, ever | `crm/followup.py` + DB constraint | 8 | TODO |
| R9 | Warnings show type + reason ; redistribution is manual only | `core/health.py` | 2 | TODO |
| R10 | Everything editable from the dashboard, no code changes | `pages/Settings.jsx` | 9 | TODO |

B. Agent tasks – spec §6 (all 17, in execution order)

| ID | Task | Detail that must not be lost | File | Phase | Status |
|------|-----------------------------|---|-------------------------------------|-------|--------|
| ---- | ----- | ----- | ----- | ----- | ----- |
| T1 | Boot & load config | Loads niche, location, language, targets, style, style duration, proxy creds + the full previously-contacted list (dedup) + the re-contact-eligible list | `main.py` | 2 | TODO |
| T2 | Health check & session open | Warning type + specific reason (not generic); pause account; wait for Hussein on redistribution; isolated context per proxy IP | `core/health.py`, `core/session.py` | 2 | TODO |
| T3 | LinkedIn discovery | Search by niche/industry/company size/location/activity; 30/account; reads company name, description, headcount, website, recent posts, public contact info; prioritises recently active over dormant ; adapts search terms if results are weak | `discovery/linkedin.py` | 3 | TODO |
| T4 | Instagram discovery | Hashtags + explore + location tags; 20/account; reads bio, followers, post count, engagement, story signals; skips inactive/irrelevant; reads only, sends nothing | `discovery/instagram.py` | 3 | TODO |
| T5 | Business qualification | Reasoning decision, not a keyword filter ; skip personal/inactive/no clear service/wrong niche | `discovery/qualify.py` | 3 | TODO |
| T6 | Deep profile analysis | Company size · revenue tier · industry · website analysis (design quality, booking flow, CTA, load speed) · ads activity · all social platforms active on · full weak points list · AI opportunities. Stored and shown in full , never summarised | `analysis/analyze.py` | 4 | TODO |
| T7 | Founder detection | Phrases: "founder of X", "co-founder", "CEO & founder", "established by", "created by", "owner of", + any equivalent wording. Stores founder name and the matched bio phrase . Dashboard shows two flags on one card (company messaged + founder found). Fires WhatsApp alert | `analysis/founder.py` | 4 | TODO |
| T8 | WhatsApp number detection | Checks bio, profile description, website link, contact section – on both platforms. Public listed numbers only, never guessed or scraped . Fires WhatsApp channel in addition to , not instead of, the primary channel | `analysis/whatsapp_detect.py` | 4 | TODO |
| T9 | Lead scoring | 1-10 from fit + pain + budget potential ; Hot 8-10 / Warm 5-7 / Cold 1-4; written reasoning always present | `analysis/score.py` | 4 | TODO |
| T10 | Save to Client History | Happens before any message is generated; permanent; saved whether or not outreach | | | |

```

ever happens | `db/repositories.py` | 4 | TODO |
| T11 | Message generation | Sonnet; human tone; greeting uses business/team/**founder** name; references that lead's
specific weak points; Direct vs Discovery style; style holds for the configured duration then rotates |
`messaging/generate.py`, `messaging/style.py` | 5 | TODO |
| T12 | Queue for approval | All of the day's messages queued; Mahmoud edits/approves/holds; nothing sends on any
channel until approved | `crm/` + `pages/ApprovalQueue.jsx` | 6 | TODO |
| T13 | Route & send | LinkedIn auto · WhatsApp auto · Instagram to manual queue. Every send logged with timestamp,
account used, platform, message text | `sending/` | 7 | TODO |
| T14 | Hot lead instant alert | Fires immediately on scoring, not at end of run. Includes name, platform, score,
reasoning, dashboard deep-link | `notifications/whatsapp_notify.py` | 8 | TODO |
| T15 | CRM record + pipeline entry | New → Contacted → Replied → Interested → Meeting Booked → Deal Closed → Lost. Auto-
move on reply; manual moves allowed; every change timestamped | `crm/pipeline.py` | 8 | TODO |
| T16 | Follow-up & re-engagement | Per-lead on/off + timing set by Hussein (agent never picks the delay); re-
engagement for replied-then-cold; pauses instantly on reply; max 2 contacts; per-lead re-contact gap |
`crm/followup.py` | 8 | TODO |
| T17 | Daily summary | WhatsApp to both numbers: leads found by platform, messages sent by platform, hot/warm/cold
breakdown, warnings that occurred, exact finish time, total duration | `notifications/whatsapp_notify.py` | 8 | TODO |

```

C. Dashboard sections – spec §7 (all 10)

```

ID	Section	Every element required	Phase	Status
D1	Daily Live Feed	Card per lead: business name, platform badge, profile photo, follower/headcount, industry, score,		
temperature badge (colour-coded), ALL weak points – full list, not hidden, AI opportunities, founder flag (two-part),				
generated message copyable in one tap, pipeline status. Hot leads highlighted and pinned to top. Filters:				
platform, score range, temperature, date. Click → Lead Detail	9	TODO		
D2	Instagram Manual Send	Separate section; shows only post-approval IG leads; name, score, temperature, copyable		
message, founder flag, "Mark as Sent" → enters pipeline as Contacted; shows pending vs sent today; must work on				
iPhone	9	TODO		
D3	Approval Section	Per item: name, platform, score, temperature, full message text, Edit + Approve; individual		
approve and Approve All; count badge ("14 awaiting"); mobile-first	9	TODO		
D4	CRM Pipeline Board	7 kanban columns; card shows name, platform, score, temperature, last activity; drag and		
drop; auto-move on reply; click → Lead Detail	9	TODO		
D5	Lead Detail	Everything: profile data, website, all social platforms, full weak points, AI opportunities, founder		
name + flag, WhatsApp number, score + full reasoning, temperature, all messages ever sent with dates, all				
replies with dates, stage + full stage history with timestamps, follow-up settings, contact count, first contact				
date, editable notes. Edit follow-up here. Move stage here	9	TODO		
D6	Client History	Permanent, never resets. Search/filter by name, industry, platform, score, temperature, date		
range, contacted y/n, founder y/n, stage. Totals at top: ever analysed, contacted, hot/warm/cold	9	TODO		
D7	Analytics & KPIs	Headline cards: businesses reached (all/month/week/today), messages sent, replies + reply		
rate %, meetings booked, deals closed, hot leads found. Charts: reached per platform (bar), per account (bar), sent vs				
pending approval, leads per stage (funnel), hot/warm/cold (donut), reply rate per platform (bar), founders detected +				
founders contacted, IG-specific (found / sent manually / pending). Daily activity timeline (line). Filters: date				
range, platform, account. Real-time updates	9	TODO		
D8	Account Health	3 accounts: Active/Warned/Paused; specific warning type + reason; per-account redistribute		
toggle (manual); usage today vs limit; last active time; proxy status: connected/missing/error	9	TODO		
D9	Run Status	Running/Idle/Error; which account is active and what task it's on; start + finish time;		
total duration ("Completed in 2h 14m"); progress "X of 150 processed"	9	TODO		
D10	Settings	Every control in section D below	9	TODO
D11	PWA	`manifest.json` + icons; installable via Safari → Add to Home Screen; full-screen on iPhone	9	TODO

```

D. Editable settings – spec §8 (nothing here may require a code change)

```

ID	Setting	Phase	Status
S1	Target niche / industry / business type	1 (schema) → 9 (UI)	TODO
S2	Target location / country / market – geography-configurable, not hardcoded	1 → 9	TODO
S3	Outreach language(s)	1 → 9	TODO
S4	Instagram leads/day per account (default 20)	1 → 9	TODO
S5	LinkedIn leads/day per account (default 30)	1 → 9	TODO
S6	Run time – per account, set independently (not one shared time)	1 → 9	TODO
S7	Active message style (Direct / Discovery / extensible)	1 → 9	TODO
S8	Style duration before rotation	1 → 9	TODO
S9	Per-lead follow-up on/off + timing (from Lead Detail)	8 → 9	TODO

```

| | | | |
|-----|--|-------|------|
| S10 | Re-contact gap per lead | 8 → 9 | TODO |
| S11 | Contact cap = 2 – **deliberately NOT user-editable** | 8 | TODO |
| S12 | Proxy IP/credentials per account × 3 – **editable slot, may be empty at launch** | 2 → 9 | TODO |
| S13 | Redistribution toggle per account – **never automatic** | 2 → 9 | TODO |
| S14 | WhatsApp recipient number 1 + number 2 | 1 → 9 | TODO |
| S15 | Optional per-type alert routing (e.g. approval reminders → Mahmoud only) – spec says "finalise during build" | 8 | OPEN |

E. WhatsApp notifications – spec §9 (all 5)

| ID | Alert | Trigger | Recipients | Must include | Phase | Status |
|----|-------------------|---|------------|--|-------|--------|
| N1 | Hot lead | Score 8-10, **immediately on scoring** | Both | name, platform, score, reasoning, dashboard link | 8 | TODO |
| N2 | Founder found | Founder detected alongside company message | Both | name, founder name, platform, dashboard link | 8 | TODO |
| N3 | Account warning | Immediately on account problem | Both | which account, warning type, **specific reason** | 8 | TODO |
| N4 | Approval reminder | Day's messages queued | Mahmoud | count waiting | 8 | TODO |
| N5 | Daily summary | Run completion | Both | leads by platform, messages by platform, hot/warm/cold, warnings, **finish time, duration** | 8 | TODO |

F. Account safety & proxy plan – spec §11

| ID | Requirement | Phase | Status |
|----|--|-------|--------|
| P1 | Sticky proxies only – **rotating proxies must never be used** | 2 | TODO |
| P2 | Residential (or mobile for IG) – **datacenter proxies must never be used** | 2/10 | TODO |
| P3 | Account → IP is a permanent 1:1 assignment; contexts never mix | 2 | TODO |
| P4 | Proxy config slot exists from day one even when empty; filling it later is **settings-only, zero code change** | 2 | TODO |
| P5 | Warm-up: start 5-10/day per account, ramp over 2-4 weeks to 20 IG + 30 LI | 10 | TODO |
| P6 | Auto warm-up schedule – increments weekly toward the configured max (`core/warmup.py`) | 2/10 | TODO |
| P7 | **Vary timing/pattern between the 3 accounts** – identical parallel behaviour re-creates the "one operator" signal even on separate IPs | 10 | TODO |

G. Explicitly OUT of scope – spec §15 (listed so they are never accidentally added)

`N/A` for this build. Revisit as a future Phase 2 project.

| ID | Excluded | Note |
|-----|--|--|
| X1 | Automated appointment setter / Calendly | Pipeline stage exists, booking is **manually logged** |
| X2 | Automated meeting-booked alerts | Manual logging only |
| X3 | Meeting preparation agent | – |
| X4 | Call summary agent | – |
| X5 | Proposal generator | – |
| X6 | Post-meeting nurture sequences (Day 1/3/5/7/14) | – |
| X7 | Customer support mode | – |
| X8 | Industry-wide campaign blasting | – |
| X9 | Meta Ad Library ad-spend detection | Basic "are they running ads" **is** in scope (T6) |
| X10 | Multi-version messaging (3 tones per lead) | One message per lead |
| X11 | Smart context-aware re-engagement on live activity | **Basic** re-engagement under follow-up controls **is** in scope (T16) |

H. Open decisions – need Hussein's answer before the phase that consumes them

| ID | Question | Needed by | Default I'll use if you don't specify |
|----|---|-----------|---------------------------------------|
| Q1 | WhatsApp sending method: Twilio, 360dialog, or the informal method? | Phase 7 | Build behind an interface so the |

```
provider is swappable; pick Twilio as the reference implementation |
Q2	WhatsApp **notifications** – same provider as sending, or separate?	Phase 8	Same provider, separate config
Q3	Reply detection for LinkedIn – poll the inbox via Playwright?	Phase 8	Poll on each run
Q4	S15 alert routing per type	Phase 8	All 5 to both numbers; approval reminder to Mahmoud only
Q5	Which market/niche for the first real run?	Phase 10	Left empty in settings, you fill it in
Q6	Supabase project – created yet? URL + keys	Phase 1	Blocks Phase 1
Q7	Claude API key	Phase 4	Blocks Phase 4
```

Deviations from the source documents (flagged, not silent)

1. ****`runs` table**** is created by ``02_SUPABASE_SCHEMA.sql`` but is missing from the Phase 1 verification list in the build plan. It is the data source for D9 (Run Status) and T17 (finish time in the daily summary). Treated as a 9th table and verified in Phase 1.
2. ****Build order conflict.**** Spec §14 bundles analysis + message generation into one step and orders approval before sending. ``01_PHASED_BUILD_PLAN.md`` splits them (Phase 4 analysis / Phase 5 messages). The phased build plan wins – the master prompt names it authoritative. No requirement is dropped by either ordering.
3. ****Repo root.**** ``03_PROJECT_STRUCTURE.md`` shows a ``nexaris-outreach-agent/`` wrapper folder. Built in place instead so the planning documents sit alongside the code rather than in a sibling directory. Structure below that level is exactly as specified.

Every environment variable, documented, no real secrets

```
# =====
# NEXARIS AI OUTREACH AGENT – environment variables
# =====
# This file is committed and contains NO real secrets. Copy the relevant section
# into agent/.env and dashboard/.env, then fill in real values. Those two files
# are gitignored and must never be committed.
#
#   cp .env.example agent/.env
#   cp .env.example dashboard/.env
#
# The "needed by" note on each block tells you which phase actually requires it –
# you do not need everything filled in to start.
# =====

# -----
# AGENT (agent/.env) – the Python agent, runs on the always-on server
# -----

# --- Supabase --- needed by: Phase 1 ---
# Project URL and the SERVICE ROLE key (Settings > API in the Supabase dashboard).
# The service role key bypasses Row Level Security. It is safe here because the
# agent runs on a trusted server. NEVER put it in the dashboard.
SUPABASE_URL=https://your-project-ref.supabase.co
SUPABASE_SERVICE_KEY=

# --- Claude API --- needed by: Phase 4 ---
# From console.anthropic.com. Haiku handles analysis/scoring/founder detection
# (high volume, cost-efficient); Sonnet writes the messages (better quality).
ANTHROPIC_API_KEY=
MODEL_ANALYSIS=claude-haiku-4-5-20251001
MODEL_MESSAGES=claude-sonnet-5

# --- WhatsApp --- needed by: Phase 7 (sending) and Phase 8 (notifications) ---
# Provider not finalised yet – see open decision Q1 in REQUIREMENTS_COVERAGE.md.
WHATSAPP_PROVIDER=twilio
WHATSAPP_API_KEY=
WHATSAPP_API_SECRET=
WHATSAPP_FROM_NUMBER=

# --- Runtime --- needed by: Phase 2 ---
# Timezone that each account's run time is interpreted in.
TIMEZONE=Asia/Beirut
# HEADLESS=false shows a visible browser – useful while building discovery in
# Phase 3. Must be true on the server.
HEADLESS=true
# Safety brake for local testing: caps leads per account regardless of the
# dashboard setting. Leave empty in production.
DEV_MAX_LEADS_PER_ACCOUNT=

# --- Proxies --- needed by: Phase 10 ---
# NOTE: proxy credentials are stored PER ACCOUNT in the Supabase `accounts` table
# and edited from the dashboard – not here. Listed only so you know where they
# live. Each account gets one dedicated sticky residential IP, never shared, and
# rotating or datacenter proxies must never be used.

# -----
# DASHBOARD (dashboard/.env) – the React PWA, deployed to Vercel
# -----
# Only VITE_-prefixed vars reach the browser. Use the ANON key here, never the
# service role key – anything in this file is publicly visible in the built app.
# Row Level Security is what protects the data.
```

```
VITE_SUPABASE_URL=https://your-project-ref.supabase.co
VITE_SUPABASE_ANON_KEY=
```

.gitignore

47 lines · text

What Git is told to never track

```
# — SECRETS — never commit —
.env
.env.local
.env*.local
agent/.env
dashboard/.env
*.pem
*.key

# — Python —
venv/
.venv/
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/
.mypy_cache/

# — Playwright —
playwright-report/
test-results/
.playwright/
browser_profiles/

# — Node / Vite —
node_modules/
dist/
dist-ssr/
.vite/
*.local

# — Claude Code personal overrides (model/effort per this project) —
.claude/settings.local.json

# — Editors / OS —
.vscode/*
!.vscode/extensions.json
.idea/
.DS_Store
Thumbs.db
desktop.ini

# — Logs —
*.log
npm-debug.log*
logs/
```


B2. Database (SQL)

02_SUPABASE_SCHEMA.sql

229 lines · sql

The full schema — 9 tables, indexes, seed data

```
-- =====
-- NEXARIS AI OUTREACH AGENT - SUPABASE SCHEMA
-- Run this in the Supabase SQL editor.
-- Walk Hussein through it table by table before running.
-- =====

-- Enable UUID generation
create extension if not exists "uuid-osspl";

-- =====
-- ACCOUNTS - the 3 outreach accounts, each with its own proxy
-- =====
create table if not exists accounts (
  id                uuid primary key default uuid_generate_v4(),
  label             text not null,           -- e.g. "Account 1"
  platform_handles  jsonb default '{}':jsonb, -- {linkedin: "...", instagram: "..."}
  run_time          time not null default '09:00', -- individual daily run time
  ig_daily_limit    int  not null default 20,    -- editable
  linkedin_daily_limit int not null default 30,  -- editable
  proxy_host        text,                     -- proxy config slot (nullable - can be empty at launch)
  proxy_port        text,
  proxy_username    text,
  proxy_password    text,
  warmup_current_limit int default 5,          -- current warm-up cap, ramps up over weeks
  status            text not null default 'active', -- active | warned | paused
  warning_type       text,                     -- e.g. "captcha", "login_challenge", "limit_reached"
  warning_reason     text,                     -- human-readable explanation
  redistribute_flag  boolean default false,    -- Hussein's manual decision when paused
  last_active_at    timestampz,
  created_at        timestampz default now()
);

-- =====
-- SETTINGS - single-row global config, editable from dashboard
-- =====
create table if not exists settings (
  id                uuid primary key default uuid_generate_v4(),
  -- targeting
  target_niche      text default '',
  target_industry   text default '',
  target_location   text default '',           -- country / market
  target_business_type text default '',
  outreach_languages text[] default array['English'],
  -- message style
  message_style     text default 'discovery',  -- direct | discovery
  style_duration_days int default 7,           -- rotate style after N days
  style_last_rotated_at timestampz default now(),
  -- follow-up / contact rules
  default_recontact_gap_days int default 30,   -- default gap before 2nd touch
  max_contacts_per_lead int default 2,         -- HARD RULE - do not exceed
  -- notifications
  whatsapp_recipient_1 text,                   -- e.g. Hussein
  whatsapp_recipient_2 text,                   -- e.g. Mahmoud
  -- approval
  approval_required boolean default true,      -- permanent hard gate
  updated_at        timestampz default now()
);
```

```

-- =====
-- LEADS – every business the agent discovers
-- =====
create table if not exists leads (
  id                uuid primary key default uuid_generate_v4(),
  account_id        uuid references accounts(id), -- which account found it
  platform          text not null,              -- linkedin | instagram
  -- profile basics
  business_name     text,
  profile_url       text,
  profile_photo_url text,
  follower_count    int,
  industry          text,
  website           text,
  social_platforms  jsonb default '[]'::jsonb,   -- all platforms they're active on
  -- deep analysis
  company_size      text,
  revenue_tier      text,
  ads_running       boolean,
  weak_points       jsonb default '[]'::jsonb,   -- full list, shown in dashboard
  ai_opportunities  jsonb default '[]'::jsonb,
  -- founder detection
  founder_found     boolean default false,
  founder_name      text,
  founder_source_phrase text,                  -- the bio phrase that matched
  -- whatsapp detection
  whatsapp_found     boolean default false,
  whatsapp_number   text,
  -- scoring
  score             int,                       -- 1-10
  temperature       text,                     -- hot | warm | cold
  score_reasoning   text,                     -- the WHY
  -- message
  generated_message text,
  message_style_used text,
  -- lifecycle
  status            text not null default 'discovered',
                      -- discovered | analyzed | awaiting_approval | approved
                      -- | manual_send_pending | contacted | replied | interested
                      -- | meeting_booked | deal_closed | lost
  contact_count     int default 0,             -- 0, 1, or 2 (max 2)
  first_contacted_at timestamptz,
  notes            text,
  created_at       timestamptz default now(),
  updated_at       timestamptz default now()
);

-- =====
-- MESSAGES – every message sent/queued per lead
-- =====
create table if not exists messages (
  id                uuid primary key default uuid_generate_v4(),
  lead_id           uuid references leads(id) on delete cascade,
  channel           text not null,             -- linkedin | whatsapp | instagram
  body             text not null,
  is_followup       boolean default false,
  is_reengagement   boolean default false,
  -- approval
  approval_status   text default 'awaiting',   -- awaiting | approved | held | edited
  approved_by       text,                     -- Mahmoud
  approved_at       timestamptz,
  edited_body       text,                     -- if Mahmoud edited it
  -- send
  send_status       text default 'pending',    -- pending | sent | manual_pending | failed
  sent_at          timestamptz,
  sent_via_account  uuid references accounts(id),
  created_at       timestamptz default now()
);

```

```

-- =====
-- PIPELINE_HISTORY – every stage change, timestamped
-- =====
create table if not exists pipeline_history (
  id          uuid primary key default uuid_generate_v4(),
  lead_id     uuid references leads(id) on delete cascade,
  from_stage  text,
  to_stage    text not null,
  changed_by  text default 'agent',          -- agent | hussein | mahmoud
  changed_at  timestampz default now()
);

-- =====
-- FOLLOW_UPS – scheduled follow-ups / re-engagements per lead
-- =====
create table if not exists follow_ups (
  id          uuid primary key default uuid_generate_v4(),
  lead_id     uuid references leads(id) on delete cascade,
  enabled     boolean default false,         -- per-lead on/off (Hussein's choice)
  scheduled_for timestampz,                 -- Hussein sets the timing
  is_reengagement boolean default false,
  status      text default 'scheduled',     -- scheduled | sent | cancelled | paused
  created_at  timestampz default now()
);

-- =====
-- CLIENT_HISTORY – permanent record of EVERY analyzed lead
-- (never resets; separate from the live feed)
-- =====
create table if not exists client_history (
  id          uuid primary key default uuid_generate_v4(),
  lead_id     uuid references leads(id) on delete set null,
  business_name text,
  platform    text,
  industry    text,
  score       int,
  temperature text,
  weak_points  jsonb,
  founder_found boolean,
  founder_name text,
  contacted    boolean default false,
  snapshot     jsonb,                    -- full lead snapshot at analysis time
  analyzed_at  timestampz default now()
);

-- =====
-- NOTIFICATIONS_LOG – every WhatsApp alert sent
-- =====
create table if not exists notifications_log (
  id          uuid primary key default uuid_generate_v4(),
  type        text not null,             -- hot_lead | founder | warning | approval_reminder | daily_summary
  recipients  text[],                   -- numbers it was sent to
  body        text,
  related_lead_id uuid references leads(id) on delete set null,
  sent_at     timestampz default now()
);

-- =====
-- RUNS – one row per daily agent run (for finish-time tracking)
-- =====
create table if not exists runs (
  id          uuid primary key default uuid_generate_v4(),
  account_id  uuid references accounts(id),
  started_at  timestampz default now(),
  finished_at timestampz,
  leads_found int default 0,
  messages_sent int default 0,

```

```

    status          text default 'running',      -- running | completed | error
    notes           text
);

-- =====
-- HELPFUL INDEXES
-- =====
create index if not exists idx_leads_status      on leads(status);
create index if not exists idx_leads_temperature on leads(temperature);
create index if not exists idx_leads_platform   on leads(platform);
create index if not exists idx_leads_account    on leads(account_id);
create index if not exists idx_messages_lead    on messages(lead_id);
create index if not exists idx_messages_approval on messages(approval_status);
create index if not exists idx_pipeline_lead    on pipeline_history(lead_id);
create index if not exists idx_followups_sched  on follow_ups(scheduled_for);

-- =====
-- SEED – insert the 3 accounts and one settings row
-- (Hussein: replace the handles later with real account info)
-- =====
insert into accounts (label, run_time, ig_daily_limit, linkedin_daily_limit, warmup_current_limit)
values
    ('Account 1', '09:00', 20, 30, 5),
    ('Account 2', '11:00', 20, 30, 5),
    ('Account 3', '13:00', 20, 30, 5)
on conflict do nothing;

insert into settings (message_style, style_duration_days, default_recontact_gap_days)
select 'discovery', 7, 30
where not exists (select 1 from settings);

-- =====
-- END OF SCHEMA
-- =====

```

Row Level Security for all 9 tables

```

-- =====
-- NEXARIS AI OUTREACH AGENT – ROW LEVEL SECURITY
-- Run this in the Supabase SQL editor AFTER 02_SUPABASE_SCHEMA.sql.
-- =====
--
-- WHY THIS FILE EXISTS
-- Creating a table does not protect it. Until Row Level Security (RLS) is
-- enabled, the access rules are wide open at the policy layer and Supabase
-- will warn you that every table is unprotected.
--
-- THE MODEL (deliberately simple – only 2 users)
-- This system has exactly two people: Hussein and Mahmoud. Both log in through
-- Supabase Auth, so both carry the built-in `authenticated` role. There is no
-- "public visitor" who should ever see a lead or a message.
--
-- So the rule for every table is the same:
-- * Enable RLS.
-- * Allow the `authenticated` role to do everything (select/insert/update/delete).
-- * Everyone else (the `anon` role) gets nothing.
--
-- The AGENT is unaffected by these policies: it connects with the SERVICE ROLE
-- key, which bypasses RLS entirely. RLS only governs the DASHBOARD, which uses
-- the anon key + a logged-in session.
--
-- If later you want finer control (e.g. Mahmoud can approve but not delete
-- leads), you replace the blanket policy on a specific table with per-command
-- policies. For now, both users are trusted owners.
-- =====

-- Helper: enable RLS and grant authenticated users full access on one table.
-- Postgres has no "apply to all tables" shortcut for policies, so we repeat the
-- same three statements per table. Kept explicit so it is easy to read and to
-- later override a single table without touching the rest.

-- ---- accounts ----
alter table accounts enable row level security;
drop policy if exists "authenticated full access" on accounts;
create policy "authenticated full access" on accounts
  for all to authenticated using (true) with check (true);

-- ---- settings ----
alter table settings enable row level security;
drop policy if exists "authenticated full access" on settings;
create policy "authenticated full access" on settings
  for all to authenticated using (true) with check (true);

-- ---- leads ----
alter table leads enable row level security;
drop policy if exists "authenticated full access" on leads;
create policy "authenticated full access" on leads
  for all to authenticated using (true) with check (true);

-- ---- messages ----
alter table messages enable row level security;
drop policy if exists "authenticated full access" on messages;
create policy "authenticated full access" on messages
  for all to authenticated using (true) with check (true);

-- ---- pipeline_history ----
alter table pipeline_history enable row level security;
drop policy if exists "authenticated full access" on pipeline_history;
create policy "authenticated full access" on pipeline_history
  for all to authenticated using (true) with check (true);

```

```

-- ---- follow_ups ----
alter table follow_ups enable row level security;
drop policy if exists "authenticated full access" on follow_ups;
create policy "authenticated full access" on follow_ups
  for all to authenticated using (true) with check (true);

-- ---- client_history ----
alter table client_history enable row level security;
drop policy if exists "authenticated full access" on client_history;
create policy "authenticated full access" on client_history
  for all to authenticated using (true) with check (true);

-- ---- notifications_log ----
alter table notifications_log enable row level security;
drop policy if exists "authenticated full access" on notifications_log;
create policy "authenticated full access" on notifications_log
  for all to authenticated using (true) with check (true);

-- ---- runs ----
alter table runs enable row level security;
drop policy if exists "authenticated full access" on runs;
create policy "authenticated full access" on runs
  for all to authenticated using (true) with check (true);

-- =====
-- VERIFY (optional): after running, this should list 9 rows, all rowsecurity=true
--   select tablename, rowsecurity from pg_tables
--   where schemaname = 'public' order by tablename;
-- =====

```

Reference copy of the schema (identical to 02_SUPABASE_SCHEMA.sql)

```
-- =====
-- NEXARIS AI OUTREACH AGENT – SUPABASE SCHEMA
-- Run this in the Supabase SQL editor.
-- Walk Hussein through it table by table before running.
-- =====

-- Enable UUID generation
create extension if not exists "uuid-osspl";

-- =====
-- ACCOUNTS – the 3 outreach accounts, each with its own proxy
-- =====
create table if not exists accounts (
  id                uuid primary key default uuid_generate_v4(),
  label             text not null,           -- e.g. "Account 1"
  platform_handles  jsonb default '{}':jsonb, -- {linkedin: "...", instagram: "..."}
  run_time          time not null default '09:00', -- individual daily run time
  ig_daily_limit    int  not null default 20,    -- editable
  linkedin_daily_limit int not null default 30,  -- editable
  proxy_host        text,                     -- proxy config slot (nullable – can be empty at launch)
  proxy_port        text,
  proxy_username    text,
  proxy_password    text,
  warmup_current_limit int default 5,          -- current warm-up cap, ramps up over weeks
  status            text not null default 'active', -- active | warned | paused
  warning_type      text,                     -- e.g. "captcha", "login_challenge", "limit_reached"
  warning_reason    text,                     -- human-readable explanation
  redistribute_flag boolean default false,     -- Hussein's manual decision when paused
  last_active_at    timestampz,
  created_at        timestampz default now()
);

-- =====
-- SETTINGS – single-row global config, editable from dashboard
-- =====
create table if not exists settings (
  id                uuid primary key default uuid_generate_v4(),
  -- targeting
  target_niche      text default '',
  target_industry   text default '',
  target_location   text default '',           -- country / market
  target_business_type text default '',
  outreach_languages text[] default array['English'],
  -- message style
  message_style     text default 'discovery',  -- direct | discovery
  style_duration_days int default 7,           -- rotate style after N days
  style_last_rotated_at timestampz default now(),
  -- follow-up / contact rules
  default_recontact_gap_days int default 30,   -- default gap before 2nd touch
  max_contacts_per_lead int default 2,         -- HARD RULE – do not exceed
  -- notifications
  whatsapp_recipient_1 text,                   -- e.g. Hussein
  whatsapp_recipient_2 text,                   -- e.g. Mahmoud
  -- approval
  approval_required boolean default true,      -- permanent hard gate
  updated_at        timestampz default now()
);

-- =====
-- LEADS – every business the agent discovers
-- =====
create table if not exists leads (
  id                uuid primary key default uuid_generate_v4(),
  account_id        uuid references accounts(id), -- which account found it
```

```

platform          text not null,                -- linkedin | instagram
-- profile basics
business_name     text,
profile_url       text,
profile_photo_url text,
follower_count    int,
industry          text,
website           text,
social_platforms  jsonb default '[]'::jsonb,    -- all platforms they're active on
-- deep analysis
company_size      text,
revenue_tier      text,
ads_running       boolean,
weak_points       jsonb default '[]'::jsonb,    -- full list, shown in dashboard
ai_opportunities  jsonb default '[]'::jsonb,
-- founder detection
founder_found     boolean default false,
founder_name      text,
founder_source_phrase text,                  -- the bio phrase that matched
-- whatsapp detection
whatsapp_found    boolean default false,
whatsapp_number   text,
-- scoring
score            int,                        -- 1-10
temperature       text,                     -- hot | warm | cold
score_reasoning   text,                     -- the WHY
-- message
generated_message text,
message_style_used text,
-- lifecycle
status           text not null default 'discovered',
                -- discovered | analyzed | awaiting_approval | approved
                -- | manual_send_pending | contacted | replied | interested
                -- | meeting_booked | deal_closed | lost
contact_count     int default 0,             -- 0, 1, or 2 (max 2)
first_contacted_at timestampz,
notes            text,
created_at       timestampz default now(),
updated_at       timestampz default now()
);

-- =====
-- MESSAGES – every message sent/queued per lead
-- =====
create table if not exists messages (
  id          uuid primary key default uuid_generate_v4(),
  lead_id     uuid references leads(id) on delete cascade,
  channel     text not null,                -- linkedin | whatsapp | instagram
  body       text not null,
  is_followup boolean default false,
  is_reengagement boolean default false,
  -- approval
  approval_status text default 'awaiting',  -- awaiting | approved | held | edited
  approved_by     text,                    -- Mahmoud
  approved_at     timestampz,
  edited_body     text,                    -- if Mahmoud edited it
  -- send
  send_status     text default 'pending',   -- pending | sent | manual_pending | failed
  sent_at        timestampz,
  sent_via_account uuid references accounts(id),
  created_at     timestampz default now()
);

-- =====
-- PIPELINE_HISTORY – every stage change, timestamped
-- =====
create table if not exists pipeline_history (
  id          uuid primary key default uuid_generate_v4(),

```



```

    lead_id          uuid references leads(id) on delete cascade,
    from_stage       text,
    to_stage         text not null,
    changed_by       text default 'agent',          -- agent | hussein | mahmoud
    changed_at       timestampz default now()
);

-- =====
-- FOLLOW_UPS – scheduled follow-ups / re-engagements per lead
-- =====
create table if not exists follow_ups (
    id               uuid primary key default uuid_generate_v4(),
    lead_id          uuid references leads(id) on delete cascade,
    enabled          boolean default false,         -- per-lead on/off (Hussein's choice)
    scheduled_for    timestampz,                   -- Hussein sets the timing
    is_reengagement  boolean default false,
    status           text default 'scheduled',      -- scheduled | sent | cancelled | paused
    created_at       timestampz default now()
);

-- =====
-- CLIENT_HISTORY – permanent record of EVERY analyzed lead
-- (never resets; separate from the live feed)
-- =====
create table if not exists client_history (
    id               uuid primary key default uuid_generate_v4(),
    lead_id          uuid references leads(id) on delete set null,
    business_name    text,
    platform         text,
    industry         text,
    score            int,
    temperature      text,
    weak_points      jsonb,
    founder_found    boolean,
    founder_name     text,
    contacted        boolean default false,
    snapshot         jsonb,                       -- full lead snapshot at analysis time
    analyzed_at      timestampz default now()
);

-- =====
-- NOTIFICATIONS_LOG – every WhatsApp alert sent
-- =====
create table if not exists notifications_log (
    id               uuid primary key default uuid_generate_v4(),
    type            text not null,                -- hot_lead | founder | warning | approval_reminder | daily_summary
    recipients       text[],                     -- numbers it was sent to
    body            text,
    related_lead_id  uuid references leads(id) on delete set null,
    sent_at         timestampz default now()
);

-- =====
-- RUNS – one row per daily agent run (for finish-time tracking)
-- =====
create table if not exists runs (
    id               uuid primary key default uuid_generate_v4(),
    account_id       uuid references accounts(id),
    started_at       timestampz default now(),
    finished_at      timestampz,
    leads_found      int default 0,
    messages_sent    int default 0,
    status           text default 'running',      -- running | completed | error
    notes           text
);

-- =====
-- HELPFUL INDEXES

```

```

-- =====
create index if not exists idx_leads_status      on leads(status);
create index if not exists idx_leads_temperature on leads(temperature);
create index if not exists idx_leads_platform   on leads(platform);
create index if not exists idx_leads_account    on leads(account_id);
create index if not exists idx_messages_lead    on messages(lead_id);
create index if not exists idx_messages_approval on messages(approval_status);
create index if not exists idx_pipeline_lead    on pipeline_history(lead_id);
create index if not exists idx_followups_sched  on follow_ups(scheduled_for);

-- =====
-- SEED – insert the 3 accounts and one settings row
-- (Hussein: replace the handles later with real account info)
-- =====
insert into accounts (label, run_time, ig_daily_limit, linkedin_daily_limit, warmup_current_limit)
values
  ('Account 1', '09:00', 20, 30, 5),
  ('Account 2', '11:00', 20, 30, 5),
  ('Account 3', '13:00', 20, 30, 5)
on conflict do nothing;

insert into settings (message_style, style_duration_days, default_recontact_gap_days)
select 'discovery', 7, 30
where not exists (select 1 from settings);

-- =====
-- END OF SCHEMA
-- =====

```

B3. Backend — Python Agent: Core

agent/config.py

69 lines · python

The single place every secret/setting is read from

```
"""
Central configuration for the agent.

Every secret and environment-specific value is read here and nowhere else, so there is
exactly one place to look when something is misconfigured. Values come from agent/.env
(never committed) – see .env.example in the repo root for the full list.
"""

import os
from pathlib import Path

from dotenv import load_dotenv

# agent/.env sits next to this file. load_dotenv is a no-op if the file doesn't exist,
# which is what we want on the server where real env vars are set by the OS instead.
AGENT_DIR = Path(__file__).parent
load_dotenv(AGENT_DIR / ".env")

def _get(key: str, default: str | None = None) -> str | None:
    """Read an env var, treating an empty string the same as missing."""
    value = os.getenv(key, default)
    return value if value else default

# — Supabase —————
# The agent uses the SERVICE_ROLE key (full read/write, bypasses RLS) because it runs
# on a trusted server. The dashboard uses the anon key instead – never swap these.
SUPABASE_URL = _get("SUPABASE_URL")
SUPABASE_SERVICE_KEY = _get("SUPABASE_SERVICE_KEY")

# — Claude API —————
ANTHROPIC_API_KEY = _get("ANTHROPIC_API_KEY")

# Model split per the spec: Haiku for the high-volume analysis work (cost-efficient),
# Sonnet for message generation (better writing quality drives reply rates).
MODEL_ANALYSIS = _get("MODEL_ANALYSIS", "claude-haiku-4-5-20251001")
MODEL_MESSAGES = _get("MODEL_MESSAGES", "claude-sonnet-5")

# — WhatsApp (sending + the 5 notification types) —————
# Provider is not finalised yet (open decision Q1 in REQUIREMENTS_COVERAGE.md).
# Phase 7 will build behind an interface so this can be swapped without code changes.
WHATSAPP_PROVIDER = _get("WHATSAPP_PROVIDER", "twilio")
WHATSAPP_API_KEY = _get("WHATSAPP_API_KEY")
WHATSAPP_API_SECRET = _get("WHATSAPP_API_SECRET")
WHATSAPP_FROM_NUMBER = _get("WHATSAPP_FROM_NUMBER")

# — Runtime —————
# Timezone that per-account run times are interpreted in.
TIMEZONE = _get("TIMEZONE", "Asia/Beirut")

# When true, Playwright shows a visible browser window. Useful while building
# discovery in Phase 3; must be false on the headless server.
HEADLESS = _get("HEADLESS", "true").lower() == "true"

# Safety brake for local testing – caps leads per account regardless of the dashboard
# setting. Leave empty in production so the dashboard limits apply.
DEV_MAX_LEADS_PER_ACCOUNT = _get("DEV_MAX_LEADS_PER_ACCOUNT")
```

```
def missing_required(keys: list[str]) -> list[str]:
    """
    Return which of the given config keys are unset.

    Each phase calls this with only the keys it actually needs, so Phase 0 can run
    with an empty .env while Phase 4 can refuse to start without a Claude API key.
    """
    return [key for key in keys if not globals().get(key)]
```

Entry point — currently the Phase 0 self-check

```

"""
Nexaris AI Outreach Agent – entry point.

Phase 0: this runs a self-check and reports what is configured and what isn't.
It does no real work yet – no browsing, no Claude calls, no sending.

From Phase 2 onward this becomes the daily run loop: for each account, at its own
scheduled time, open a session and work through Tasks 1-17 from the spec.

Run it with:  python -m agent.main
"""

import sys

from agent import config
from agent.db import client as db

def _status(ok: bool) -> str:
    """Green tick / grey dash, so the output is scannable at a glance."""
    return "[ok] " if ok else "[--] "

def self_check() -> bool:
    """
    Report which parts of the system are configured.

    Nothing here is fatal in Phase 0 – an empty .env is the expected state right after
    setup. The point is to show Hussein exactly what still needs filling in and which
    phase will need it.
    """
    print("=" * 62)
    print("  NEXARIS AI OUTREACH AGENT - setup self-check")
    print("=" * 62)
    print()

    checks = [
        ("Python runtime", True, f"Python {sys.version.split()[0]}"),
        (
            "Supabase credentials",
            db.is_configured(),
            "set" if db.is_configured() else "not set - needed for Phase 1",
        ),
        (
            "Claude API key",
            bool(config.ANTHROPIC_API_KEY),
            "set" if config.ANTHROPIC_API_KEY else "not set - needed for Phase 4",
        ),
        (
            "WhatsApp credentials",
            bool(config.WHATSAPP_API_KEY),
            "set" if config.WHATSAPP_API_KEY else "not set - needed for Phase 7",
        ),
    ]

    for label, ok, detail in checks:
        print(f"  {_status(ok)}{label:<24} {detail}")

    # Only attempt a live connection when credentials exist, otherwise the error is
    # just noise repeating what the check above already said.
    if db.is_configured():
        ok, message = db.check_connection()
        print(f"  {_status(ok)}{'Database reachable':<24} {message}")

```

```

print()
print(f"  Models:      analysis={config.MODEL_ANALYSIS}")
print(f"           messages={config.MODEL_MESSAGES}")
print(f"  Timezone:    {config.TIMEZONE}")
print(f"  Headless:    {config.HEADLESS}")
print()
print("-" * 62)
print("  Phase 0 complete: the agent runs. It does no real work yet.")
print("  Next: Phase 1 - create the database schema in Supabase.")
print("-" * 62)

return True

def main() -> int:
    """Entry point. Returns a process exit code."""
    self_check()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

agent/requirements.txt

32 lines · text

Python dependencies

```

# — Nexaris Outreach Agent — Python dependencies —
# Install:  pip install -r requirements.txt
#           playwright install chromium

# Pins are minimum-version (>=) rather than exact (==) because supabase and anthropic
# each constrain httpx to their own range — exact pins made the set unresolvable.

# Database — Supabase (Postgres + Auth + Realtime)
supabase>=2.10.0

# Claude API — Haiku for analysis/scoring, Sonnet for message generation
anthropic>=0.42.0

# Browser automation — LinkedIn/Instagram discovery + LinkedIn sending
playwright>=1.49.1

# Environment variables from agent/.env
python-dotenv>=1.0.1

# Per-account scheduling (each account runs at its own time)
APScheduler>=3.11.0

# HTTP — website analysis, WhatsApp API calls.
# Unpinned: supabase and anthropic both depend on it and pick a compatible version.
httpx

# HTML parsing for website weak-point analysis (Phase 4)
beautifulsoup4>=4.12.3

# Timezone handling for per-account run times
pytz>=2024.2

```

Stub — Phase 2

```
"""  
Runs each account at its own configured time.  
  
Built in Phase 2. Not implemented yet.  
  
Each of the 3 accounts has an independent run time, staggered through the day  
and editable from the dashboard. Opens a row in `runs` at start and closes it  
with the finish time -- that's what the dashboard Run Status panel displays.  
"""
```

B4. Backend — Python Agent: Database Layer

agent/db/client.py

63 lines · python

Lazy Supabase client, service-role key

```
"""
Supabase connection for the agent.

One shared client for the whole process. Created lazily on first use so that importing
this module never fails when credentials aren't set yet (Phase 0 runs without them).
"""

from supabase import Client, create_client

from agent import config

# Module-level cache – create_client opens a connection pool, so we make exactly one.
_client: Client | None = None

class SupabaseNotConfigured(RuntimeError):
    """Raised when the agent needs the database but credentials are missing."""

def get_client() -> Client:
    """
    Return the shared Supabase client, creating it on first call.

    Uses the service role key: the agent runs on a trusted server and needs full
    read/write access across all tables, bypassing Row Level Security.
    """
    global _client

    if _client is None:
        missing = config.missing_required(["SUPABASE_URL", "SUPABASE_SERVICE_KEY"])
        if missing:
            raise SupabaseNotConfigured(
                f"Missing {'', '.join(missing)} in agent/.env – "
                "copy .env.example and fill in your Supabase project details."
            )
        _client = create_client(config.SUPABASE_URL, config.SUPABASE_SERVICE_KEY)

    return _client

def is_configured() -> bool:
    """True when Supabase credentials are present. Lets callers degrade gracefully."""
    return not config.missing_required(["SUPABASE_URL", "SUPABASE_SERVICE_KEY"])

def check_connection() -> tuple[bool, str]:
    """
    Verify the agent can actually reach the database.

    Returns (ok, message) rather than raising, because this is used by the Phase 0
    health check where a failure is informational, not fatal.
    """
    if not is_configured():
        return False, "Supabase credentials not set in agent/.env"

    try:
        # settings is a single-row table that exists from the schema onward. Selecting
        # one row is the cheapest possible proof that the connection and keys work.

```



```
get_client().table("settings").select("id").limit(1).execute()
return True, "Connected to Supabase"
except Exception as exc: # noqa: BLE001 – surface any connection problem verbatim
return False, f"Supabase connection failed: {exc}"
```

The data-access layer — 25 functions across all 9 tables

"""

Data-access layer – the ONLY place that reads or writes Supabase tables.

Every other module in the agent goes through the functions here instead of touching the database directly. Two payoffs:

1. If the schema changes, this one file changes – not every caller.
2. Callers read like plain English: ``repositories.insert_lead(...)``, ``repositories.get_settings()``. No SQL leaks into the business logic.

The module is organised one section per table, in the same order as 02_SUPABASE_SCHEMA.sql: accounts, settings, leads, messages, pipeline_history, follow_ups, client_history, notifications_log, runs.

All functions use the shared service-role client from db/client.py, so they run on the trusted server with full access and bypass Row Level Security.

"""

from __future__ import annotations

from typing import Any

from agent.db.client import get_client

Type alias: a database row is just a dict of column -> value.

Row = dict[str, Any]

=====

ACCOUNTS – the 3 outreach accounts, each with its own proxy slot

=====

def list_accounts() -> list[Row]:

"""All accounts, oldest first. Used by the account pool at run start."""

res = get_client().table("accounts").select("*").order("created_at").execute()

return res.data

def get_account(account_id: str) -> Row | None:

"""One account by id, or None if it doesn't exist."""

res = get_client().table("accounts").select("*").eq("id", account_id).limit(1).execute()

return res.data[0] if res.data else None

def update_account(account_id: str, fields: Row) -> Row:

"""Patch one account (e.g. status, warning fields, proxy creds, warm-up cap)."""

res = get_client().table("accounts").update(fields).eq("id", account_id).execute()

return res.data[0]

def set_account_warning(account_id: str, warning_type: str, reason: str) -> Row:

"""

Record an account warning and pause it.

Core rule R9: warnings carry a specific TYPE and REASON, never generic, and the account is paused. Quota is never redistributed automatically – that stays a manual decision, so we only raise the flag, we don't act on it.

"""

return update_account(account_id, {
 "status": "paused",
 "warning_type": warning_type,
 "warning_reason": reason,
 })

```

# =====
# SETTINGS – single global row, edited from the dashboard
# =====

def get_settings() -> Row | None:
    """
    The one settings row. The agent loads this at the start of every run (Task 1)
    to pick up niche, location, language, targets, message style, and so on.
    """
    res = get_client().table("settings").select("*").limit(1).execute()
    return res.data[0] if res.data else None

def update_settings(fields: Row) -> Row:
    """Patch the settings row. `id` is looked up so callers don't have to pass it."""
    current = get_settings()
    if current is None:
        raise RuntimeError("No settings row exists – run the schema seed first.")
    res = get_client().table("settings").update(fields).eq("id", current["id"]).execute()
    return res.data[0]

# =====
# LEADS – every business the agent discovers
# =====

def insert_lead(lead: Row) -> Row:
    """Insert a freshly discovered lead. Returns the row with its generated id."""
    res = get_client().table("leads").insert(lead).execute()
    return res.data[0]

def update_lead(lead_id: str, fields: Row) -> Row:
    """Patch a lead (analysis results, score, generated message, status, ...)."""
    res = get_client().table("leads").update(fields).eq("id", lead_id).execute()
    return res.data[0]

def get_lead(lead_id: str) -> Row | None:
    res = get_client().table("leads").select("*").eq("id", lead_id).limit(1).execute()
    return res.data[0] if res.data else None

def leads_by_status(status: str) -> list[Row]:
    """All leads in a given lifecycle status (e.g. 'awaiting_approval')."""
    res = get_client().table("leads").select("*").eq("status", status).execute()
    return res.data

def contacted_profile_urls() -> set[str]:
    """
    Profile URLs of every lead already contacted at least once.

    Task 1 deduplication: loaded at run start so no business is ever contacted
    twice unintentionally. Returned as a set for O(1) membership checks during
    discovery.
    """
    res = (
        get_client()
        .table("leads")
        .select("profile_url")
        .gt("contact_count", 0)
        .execute()
    )
    return {r["profile_url"] for r in res.data if r.get("profile_url")}

```

```

# =====
# MESSAGES – every message queued/sent per lead
# =====

def insert_message(message: Row) -> Row:
    res = get_client().table("messages").insert(message).execute()
    return res.data[0]

def update_message(message_id: str, fields: Row) -> Row:
    res = get_client().table("messages").update(fields).eq("id", message_id).execute()
    return res.data[0]

def messages_awaiting_approval() -> list[Row]:
    """
    Everything Mahmoud still needs to approve. Feeds the approval queue badge and
    the hard send-gate (nothing sends while this is non-empty for a given lead).
    """
    res = (
        get_client()
        .table("messages")
        .select("*")
        .eq("approval_status", "awaiting")
        .execute()
    )
    return res.data

def messages_for_lead(lead_id: str) -> list[Row]:
    res = (
        get_client()
        .table("messages")
        .select("*")
        .eq("lead_id", lead_id)
        .order("created_at")
        .execute()
    )
    return res.data

# =====
# PIPELINE_HISTORY – every stage change, timestamped audit trail
# =====

def record_stage_change(
    lead_id: str, from_stage: str | None, to_stage: str, changed_by: str = "agent"
) -> Row:
    """
    Append one row to the audit trail. Called by pipeline.py whenever a lead moves,
    whether the move was automatic (reply detected) or manual (drag on the board).
    """
    res = get_client().table("pipeline_history").insert({
        "lead_id": lead_id,
        "from_stage": from_stage,
        "to_stage": to_stage,
        "changed_by": changed_by,
    }).execute()
    return res.data[0]

def stage_history(lead_id: str) -> list[Row]:
    """Full ordered stage history for the Lead Detail page."""
    res = (
        get_client()
        .table("pipeline_history")
        .select("*")
        .eq("lead_id", lead_id)
    )

```

```

        .order("changed_at")
        .execute()
    )
    return res.data

# =====
# FOLLOW_UPS – per-lead follow-up / re-engagement schedule
# =====

def insert_follow_up(follow_up: Row) -> Row:
    res = get_client().table("follow_ups").insert(follow_up).execute()
    return res.data[0]

def update_follow_up(follow_up_id: str, fields: Row) -> Row:
    res = get_client().table("follow_ups").update(fields).eq("id", follow_up_id).execute()
    return res.data[0]

def due_follow_ups(now_iso: str) -> list[Row]:
    """
    Enabled follow-ups whose scheduled time has arrived and are still pending.
    The agent runs these at exactly the time Hussein set per lead.
    """
    res = (
        get_client()
        .table("follow_ups")
        .select("")
        .eq("enabled", True)
        .eq("status", "scheduled")
        .lte("scheduled_for", now_iso)
        .execute()
    )
    return res.data

# =====
# CLIENT_HISTORY – permanent record of every analysed lead (never resets)
# =====

def insert_client_history(entry: Row) -> Row:
    """
    Save a permanent snapshot. Task 10: this happens BEFORE any message is
    generated, so a lead lives here forever whether or not it is ever contacted.
    """
    res = get_client().table("client_history").insert(entry).execute()
    return res.data[0]

def mark_client_history_contacted(lead_id: str) -> None:
    """Flip the `contacted` flag once outreach actually goes out for this lead."""
    get_client().table("client_history").update({"contacted": True}).eq(
        "lead_id", lead_id
    ).execute()

# =====
# NOTIFICATIONS_LOG – every WhatsApp alert sent
# =====

def log_notification(
    notif_type: str, recipients: list[str], body: str, related_lead_id: str | None = None
) -> Row:
    """
    Record a sent alert. One of the 5 types: hot_lead, founder, warning,
    approval_reminder, daily_summary.
    """

```

```

res = get_client().table("notifications_log").insert({
    "type": notif_type,
    "recipients": recipients,
    "body": body,
    "related_lead_id": related_lead_id,
}).execute()
return res.data[0]

# =====
# RUNS – one row per account per daily run (finish-time tracking)
# =====

def start_run(account_id: str) -> Row:
    """
    Open a run row when an account begins. `started_at` defaults to now() in the
    schema. Returns the row so the scheduler can close it later with the id.
    """
    res = get_client().table("runs").insert({
        "account_id": account_id,
        "status": "running",
    }).execute()
    return res.data[0]

def finish_run(
    run_id: str,
    leads_found: int,
    messages_sent: int,
    status: str = "completed",
    finished_at_iso: str | None = None,
    notes: str | None = None,
) -> Row:
    """
    Close a run with its results and finish time. The finish time feeds both the
    dashboard Run Status panel and the daily WhatsApp summary.
    """
    fields: Row = {
        "leads_found": leads_found,
        "messages_sent": messages_sent,
        "status": status,
    }
    if finished_at_iso is not None:
        fields["finished_at"] = finished_at_iso
    if notes is not None:
        fields["notes"] = notes
    res = get_client().table("runs").update(fields).eq("id", run_id).execute()
    return res.data[0]

```

The Phase 1 checkpoint test

```

"""
Phase 1 checkpoint proof – verifies the database is real and read/write works.

Run this AFTER creating the Supabase project, filling agent/.env, and running
both SQL files (02_SUPABASE_SCHEMA.sql then database/policies.sql):

    agent/venv/Scripts/python.exe -m agent.db.smoke_test

It checks, in order:
1. Connection works (credentials + network).
2. All 9 tables exist and are queryable.
3. The seed data is present (3 accounts + 1 settings row).
4. A round-trip write works: insert a throwaway lead, read it back, delete it.

Step 4 cleans up after itself, so running this leaves the database exactly as it
was. Nothing here touches your real accounts or sends anything.
"""

from __future__ import annotations

from agent.db import repositories as repo
from agent.db.client import check_connection, get_client

# Every table the schema creates. `runs` is included even though the Phase 1
# checklist omitted it – it's the data source for the Run Status panel.
TABLES = [
    "accounts", "settings", "leads", "messages", "pipeline_history",
    "follow_ups", "client_history", "notifications_log", "runs",
]

PASS = "[PASS]"
FAIL = "[FAIL]"

def _line(ok: bool, label: str, detail: str = "") -> bool:
    """Print one result line and return ok so callers can accumulate failures."""
    print(f" {PASS if ok else FAIL} {label}" + (f" – {detail}" if detail else ""))
    return ok

def run() -> int:
    print("=" * 60)
    print(" PHASE 1 SMOKE TEST – database verification")
    print("=" * 60)
    print()

    ok = True

    # ---- 1. Connection ----
    connected, message = check_connection()
    ok &= _line(connected, "Connection", message)
    if not connected:
        print("\n Cannot continue without a connection. Check agent/.env.")
        return 1

    # ---- 2. All 9 tables exist ----
    print("\n Tables:")
    client = get_client()
    for table in TABLES:
        try:
            client.table(table).select("*, count='exact'").limit(0).execute()
            _line(True, table)
        except Exception as exc: # noqa: BLE001
            ok &= _line(False, table, str(exc))

```

```

# ---- 3. Seed data present ----
print("\n Seed data:")
accounts = repo.list_accounts()
ok &= _line(len(accounts) == 3, "3 accounts seeded", f"found {len(accounts)}")
settings = repo.get_settings()
ok &= _line(settings is not None, "settings row seeded",
            f"style={settings.get('message_style')} if settings else 'MISSING'")

# ---- 4. Round-trip write/read/delete ----
print("\n Read/write round-trip:")
test_lead = None
try:
    test_lead = repo.insert_lead({
        "platform": "linkedin",
        "business_name": "SMOKE TEST – safe to ignore",
        "status": "discovered",
    })
    ok &= _line(bool(test_lead.get("id")), "insert test lead", test_lead.get("id", ""))

    read_back = repo.get_lead(test_lead["id"])
    ok &= _line(
        read_back is not None and read_back["business_name"] == test_lead["business_name"],
        "read it back",
    )
finally:
    # Always clean up, even if a check above failed mid-way.
    if test_lead and test_lead.get("id"):
        get_client().table("leads").delete().eq("id", test_lead["id"]).execute()
        _line(True, "deleted test lead (DB left clean)")

print()
print("-" * 60)
if ok:
    print(" ALL CHECKS PASSED – Phase 1 database is ready.")
else:
    print(" SOME CHECKS FAILED – see the [FAIL] lines above.")
print("-" * 60)
return 0 if ok else 1

if __name__ == "__main__":
    raise SystemExit(run())

```


B5. Backend — Python Agent: Stub Modules (future phases)

agent/core/account_pool.py

9 lines · python

Stub — Phase 2

```
"""
Loads the 3 outreach accounts and manages selection by run time.

Built in Phase 2. Not implemented yet.

Each account carries its own run time, IG/LinkedIn daily limits, warm-up cap,
and proxy slot (which may be empty). One agent, one pool -- not 3 agents.
"""
```

agent/core/session.py

10 lines · python

Stub — Phase 2

```
"""
One isolated Playwright browser context per account, on that account's proxy.

Built in Phase 2. Not implemented yet.

HARD RULE: contexts and proxy IPs never mix between accounts. Each account is
permanently assigned one dedicated sticky residential IP. Rotating proxies are
never used -- a new IP per request looks like a bot.
"""
```

agent/core/health.py

10 lines · python

Stub — Phase 2

```
"""
Detects account problems and records warning TYPE plus specific REASON.

Built in Phase 2. Not implemented yet.

Warnings must be specific ('CAPTCHA triggered', 'Login challenge detected'),
never generic. On warning: pause the account and set the redistribute flag for
Hussein to decide. Quota is NEVER redistributed automatically.
"""
```

agent/core/warmup.py

10 lines · python

Stub — Phase 2

```
"""
Per-account warm-up ramp: start at 5-10/day, climb weekly to the full target.

Built in Phase 2. Not implemented yet.

Steady state is 20 Instagram + 30 LinkedIn per account. Starting there on day
one gets accounts banned. Timing is also varied between accounts so the three
don't look like one operator.
"""
```

Stub — Phase 3

```
"""
LinkedIn search and profile reading. Target 30/account/day.

Built in Phase 3. Not implemented yet.

Reads company name, description, headcount, website, recent posts, public
contact info. Prioritises recently active companies over dormant ones.
Adapts its search terms when results come back weak -- agentic, not fixed.
"""
```

Stub — Phase 3

```
"""
Instagram search and profile reading. Target 20/account/day. READ ONLY.

Built in Phase 3. Not implemented yet.

Hashtags, explore pages, location tags. Reads bio, followers, post count,
engagement, story signals. This module NEVER sends -- Instagram's ToS bans
automated cold outreach and it risks the account.
"""
```

Stub — Phase 3

```
"""
Decides whether a discovered profile is a real business worth pursuing.

Built in Phase 3. Not implemented yet.

A reasoning decision, not a keyword filter. Skips personal accounts, inactive
pages, anything with no clear product or service, and obvious niche misses.
"""
```

Stub — Phase 4

```
"""
Deep company analysis via Claude Haiku.

Built in Phase 4. Not implemented yet.

Company size, revenue tier, industry, website analysis (design, booking flow,
CTA, load speed), ads activity, all social platforms, the FULL weak points
list, and AI opportunities. Stored in full -- never summarised or truncated.
"""
```

agent/analysis/founder.py

10 lines · python

Stub — Phase 4

```
"""
Founder detection from profile bios.

Built in Phase 4. Not implemented yet.

Matches 'founder of X', 'co-founder', 'CEO & founder', 'established by',
'created by', 'owner of', and equivalents. Stores the founder name AND the
phrase that matched. Founder outreach itself is always manual.
"""
```

agent/analysis/whatsapp_detect.py

10 lines · python

Stub — Phase 4

```
"""
Finds a public WhatsApp number in bio, description, website, or contact info.

Built in Phase 4. Not implemented yet.

Only publicly listed business numbers -- never guessed, never scraped from
private sources. A hit opens WhatsApp as an ADDITIONAL channel alongside the
primary one, not as a replacement.
"""
```

agent/analysis/score.py

10 lines · python

Stub — Phase 4

```
"""
Scores every qualified lead 1-10 with written reasoning.

Built in Phase 4. Not implemented yet.

Components: fit (match to Nexaris services), pain (number and severity of
solvable problems), budget potential (revenue signals). Hot 8-10 / Warm 5-7 /
Cold 1-4. The written WHY is always present -- it was an original requirement.
"""
```

agent/analysis/prompts.py

10 lines · python

Stub — Phase 4

```
"""
All Claude prompt templates, structured for prompt caching.

Built in Phase 4. Not implemented yet.

The long instruction blocks are identical across every lead, so they are marked
as cacheable and only the per-lead data varies. This is the main cost lever on
an estimated 4,500 leads/month.
"""
```

Stub — Phase 5

```
"""
Personalised message generation via Claude Sonnet.

Built in Phase 5. Not implemented yet.

Must read as human, not AI. Greeting uses whichever name exists: business,
team, or founder. Body references that specific lead's weak points. Tone adapts
per platform -- LinkedIn more formal, WhatsApp more direct.
"""
```

Stub — Phase 5

```
"""
Message style selection and rotation.

Built in Phase 5. Not implemented yet.

Direct style names the weak points outright. Discovery style is softer and
curiosity-driven. The active style holds for the configured duration, then
rotates. Both are editable from the dashboard at any time.
"""
```

Stub — Phase 7

```
"""
Sends approved messages on LinkedIn via the account's own session.

Built in Phase 7. Not implemented yet.

Logs timestamp, account used, platform, and message text. Moves the lead to
Contacted. Respects the account's daily limit and current warm-up cap.
"""
```

Stub — Phase 7

```
"""
Sends approved messages on WhatsApp when a public number was found.

Built in Phase 7. Not implemented yet.

Fires for leads from either platform. Provider is swappable (Twilio /
360dialog / other) -- open decision Q1.
"""
```

Stub — Phase 7

```
"""
Pushes approved Instagram messages to the dashboard's manual-send queue.

Built in Phase 7. Not implemented yet.

THIS MODULE NEVER SENDS. It marks messages 'manual_send_pending' so a human
sends them by hand and taps 'Mark as Sent', which enters the lead into the
pipeline exactly as an automated LinkedIn send would.
"""
```

Stub — Phase 8

```
"""
Pipeline stage movement and timestamped history.

Built in Phase 8. Not implemented yet.

New -> Contacted -> Replied -> Interested -> Meeting Booked -> Deal Closed ->
Lost. The agent auto-moves on send and on reply detection; everything else is
manual. Every single move is timestamped for the audit trail.
"""
```

Stub — Phase 8

```
"""
Follow-up and re-engagement scheduling. Enforces the 2-contact maximum.

Built in Phase 8. Not implemented yet.

Follow-up is a PER-LEAD decision -- Hussein sets on/off and the exact timing
himself; the agent never picks a delay. Re-engagement covers leads who replied
then went cold. Everything pauses the instant a lead replies. Max 2 contacts.
"""
```

Stub — Phase 8

```
"""
Checks LinkedIn and WhatsApp for replies and advances the pipeline.

Built in Phase 8. Not implemented yet.

A detected reply moves the lead Contacted -> Replied and immediately cancels
any scheduled follow-up or re-engagement for that lead.
"""
```

Stub — Phase 8

```
"""
The 5 WhatsApp alert types, sent to the configured recipient numbers.

Built in Phase 8. Not implemented yet.

1. Hot lead (score 8-10) -- fires immediately on scoring, not at end of run.
2. Founder found. 3. Account warning (with type and reason). 4. Approval
reminder to Mahmoud. 5. Daily summary with exact finish time and duration.
"""
```

B6. Backend — Package Init Files (9)

agent/__init__.py

2 lines · python

```
"""Nexaris AI Outreach Agent – Python agent package."""
```

agent/core/__init__.py

2 lines · python

```
"""Account pool, browser sessions, health checks, warm-up ramp. (Phase 2)"""
```

agent/discovery/__init__.py

2 lines · python

```
"""LinkedIn + Instagram lead discovery and qualification. (Phase 3)"""
```

agent/analysis/__init__.py

2 lines · python

```
"""Claude-powered analysis, founder detection, scoring. (Phase 4)"""
```

agent/messaging/__init__.py

2 lines · python

```
"""Claude Sonnet message generation and style rotation. (Phase 5)"""
```

agent/sending/__init__.py

2 lines · python

```
"""Channel routing: LinkedIn auto, WhatsApp auto, Instagram manual. (Phase 7)"""
```

agent/crm/__init__.py

2 lines · python

```
"""Pipeline, follow-up scheduling, reply detection. (Phase 8)"""
```

agent/notifications/__init__.py

2 lines · python

```
"""The 5 WhatsApp alert types. (Phase 8)"""
```

agent/db/__init__.py

2 lines · python

```
"""Supabase connection and per-table read/write functions."""
```

B7. Frontend — Dashboard Configuration

dashboard/package.json

25 lines · json

Dependencies and scripts

```
{
  "name": "nexaris-dashboard",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "description": "Nexaris AI Outreach Agent – responsive PWA dashboard (laptop + iPhone)",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.47.10",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^7.1.1"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "tailwindcss": "^4.0.0",
    "vite": "^6.0.7"
  }
}
```

dashboard/vite.config.js

23 lines · javascript

Build configuration

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  // Tailwind v4 runs as a Vite plugin – no tailwind.config.js or postcss.config.js
  // needed. Theme customisation lives in src/index.css via @theme.
  plugins: [react(), tailwindcss()],
  css: {
    // Empty inline PostCSS config. Without this, Vite walks UP the folder tree
    // looking for a postcss.config.js and finds an unrelated Tailwind v3 project
    // sitting on the Desktop, which breaks the build. Declaring it here stops the
    // search and keeps this app self-contained wherever the folder lives.
    postcss: {},
  },
  server: {
    // Exposes the dev server on the local network so the dashboard can be opened
    // on an iPhone during development, not just on the laptop.
    host: true,
    port: 5173,
  },
})
```


PWA manifest

```
{
  "name": "Nexaris Outreach Agent",
  "short_name": "Nexaris",
  "description": "Lead discovery, approval, and CRM pipeline for Nexaris outreach",
  "start_url": "/",
  "display": "standalone",
  "orientation": "portrait",
  "background_color": "#020617",
  "theme_color": "#0f172a",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

B8. Frontend — Dashboard Application Code

dashboard/index.html

26 lines · html

Page shell, PWA meta tags

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/icons/icon.svg" />

    <!-- viewport-fit=cover lets the app draw into the iPhone safe areas when it's
         launched full-screen from the home screen -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />

    <!-- PWA: these three are what make "Add to Home Screen" open full-screen on iOS -->
    <link rel="manifest" href="/manifest.json" />
    <meta name="apple-mobile-web-app-capable" content="yes" />
    <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
    <meta name="apple-mobile-web-app-title" content="Nexaris" />
    <link rel="apple-touch-icon" href="/icons/icon-180.png" />

    <meta name="theme-color" content="#0f172a" />
    <title>Nexaris Outreach</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

dashboard/src/main.jsx

11 lines · jsx

Mounts React into the page

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

The Phase 0 status page

```

import { isConfigured } from './lib/supabase'

/**
 * Phase 0 placeholder.
 *
 * Confirms the dashboard builds, Tailwind works, and Supabase credentials are wired
 * up. The real sections (live feed, approval queue, pipeline board, etc.) arrive in
 * Phase 9 – they're listed below so the build order is visible.
 */

// The 10 sections from spec §7, each mapped to the file that will hold it.
const SECTIONS = [
  ['Daily Live Feed', 'Today's leads with full weak points and copyable message'],
  ['Instagram Manual Send', 'Approved IG messages a human sends by hand'],
  ['Mahmoud's Approval', 'Edit + approve – nothing sends until this is cleared'],
  ['CRM Pipeline Board', 'Kanban across the 7 stages, drag and drop'],
  ['Lead Detail', 'Complete record, editable follow-up and notes'],
  ['Client History', 'Permanent searchable database of every lead analysed'],
  ['Analytics & KPIs', 'Every metric, filterable by date, platform, account'],
  ['Account Health', '3 accounts – status, warning type and reason, proxy state'],
  ['Run Status', 'Running / idle, current task, start and finish time, duration'],
  ['Settings', 'Every control editable without touching code'],
]

function StatusRow({ ok, label, detail }) {
  return (
    <div className="flex items-start gap-3 py-2">
      <span
        className={`mt-1.5 h-2 w-2 shrink-0 rounded-full ${
          ok ? 'bg-emerald-500' : 'bg-slate-600'
        }`}
        aria-hidden="true"
      />
      <div className="min-w-0">
        <p className="text-sm font-medium text-slate-200">{label}</p>
        <p className="text-sm text-slate-400">{detail}</p>
      </div>
    </div>
  )
}

export default function App() {
  return (
    <div className="min-h-full bg-slate-950 text-slate-100">
      <div className="mx-auto max-w-3xl px-5 py-10 sm:px-8 sm:py-14">
        <header>
          <p className="text-xs font-semibold uppercase tracking-widest text-slate-500">
            Nexaris
          </p>
          <h1 className="mt-2 text-2xl font-semibold sm:text-3xl">
            AI Outreach Agent
          </h1>
          <p className="mt-3 text-slate-400">
            Phase 0 – setup complete. The dashboard builds and runs; no real data yet.
          </p>
        </header>

        <section className="mt-10 rounded-xl border border-slate-800 bg-slate-900/50 p-5">
          <h2 className="text-sm font-semibold uppercase tracking-wide text-slate-400">
            Setup status
          </h2>
          <div className="mt-3 divide-y divide-slate-800">
            <StatusRow ok label="Dashboard" detail="React + Tailwind building correctly" />
            <StatusRow

```

```

      ok={isConfigured}
      label="Supabase connection"
      detail={
        isConfigured
        ? 'Credentials found in dashboard/.env'
        : 'Not configured – add VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY to dashboard/.env (Phase 1)'
      }
    />
  </div>
</section>

<section className="mt-8">
  <h2 className="text-sm font-semibold uppercase tracking-wide text-slate-400">
    Coming in Phase 9
  </h2>
  <ol className="mt-3 space-y-2.5">
    {SECTIONS.map(([name, detail], i) => (
      <li
        key={name}
        className="flex gap-3 rounded-lg border border-slate-800/70 bg-slate-900/30 px-4 py-3"
      >
        <span className="w-5 shrink-0 text-sm tabular-nums text-slate-600">
          {i + 1}
        </span>
        <div className="min-w-0">
          <p className="text-sm font-medium text-slate-200">{name}</p>
          <p className="text-sm text-slate-500">{detail}</p>
        </div>
      </li>
    ))}
  </ol>
</section>

<footer className="mt-10 border-t border-slate-800 pt-5 text-sm text-slate-500">
  Next up: Phase 1 – create the database schema in Supabase.
</footer>
</div>
</div>
)
}

```

Tailwind import, theme, safe-area CSS

```
@import "tailwindcss";

/* Tailwind v4 theme customisation. Extend here rather than in a config file. */
@theme {
  /* Temperature colours – used by the Hot/Warm/Cold badges across the whole
  dashboard (live feed cards, pipeline cards, approval queue, client history).
  Defined once so every screen agrees on what "hot" looks like. */
  --color-hot: oklch(0.63 0.22 25);
  --color-warm: oklch(0.75 0.16 70);
  --color-cold: oklch(0.68 0.11 235);
}

html,
body,
#root {
  height: 100%;
}

body {
  /* Safe-area padding so content clears the notch and home indicator when the
  PWA is launched full-screen from Mahmoud's iPhone home screen. */
  padding: env(safe-area-inset-top) env(safe-area-inset-right)
    env(safe-area-inset-bottom) env(safe-area-inset-left);

  /* Stops iOS Safari from auto-enlarging text in landscape. */
  -webkit-text-size-adjust: 100%;
}
```

Browser Supabase client, anon key

```
import { createClient } from '@supabase/supabase-js'

/**
 * Supabase client for the dashboard.
 *
 * Uses the ANON key, never the service role key. The anon key is safe to ship in
 * frontend code because Row Level Security (set up in Phase 1) is what actually
 * restricts access – only Hussein and Mahmoud can read or write.
 *
 * The agent uses the service role key instead. Never mix the two up.
 */

const url = import.meta.env.VITE_SUPABASE_URL
const anonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

/** True when the dashboard has credentials. Lets the UI show a helpful setup message
 * instead of crashing on a blank .env, which is the expected state in Phase 0. */
export const isConfigured = Boolean(url && anonKey)

export const supabase = isConfigured
  ? createClient(url, anonKey, {
    // Realtime keeps the live feed, run status, and analytics updating as the
    // agent works, without the dashboard polling.
    realtime: { params: { eventsPerSecond: 5 } },
  })
  : null
```

B9. Tooling

tools/make_pdf.py

72 lines · python

Renders these phase documents to PDF

```
"""
HTML -> PDF renderer for the phase documentation.

Every phase ends with a document in docs/ explaining what was built. This script
turns the HTML source into a PDF using the Chromium that Playwright already
installs for the agent, so there is no extra dependency to manage.

Usage:
    agent/venv/Scripts/python.exe tools/make_pdf.py docs/PHASE_0_EXPLAINED.html

The PDF is written next to the HTML with the same name.
"""

import sys
from pathlib import Path

from playwright.sync_api import sync_playwright


def render(html_path: Path) -> Path:
    """Render one HTML file to a PDF beside it. Returns the PDF path."""
    if not html_path.exists():
        raise FileNotFoundError(html_path)

    pdf_path = html_path.with_suffix(".pdf")

    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        # file:// so relative assets resolve. wait_until="load" makes sure fonts and
        # any images are in place before we snapshot the page.
        page.goto(html_path.resolve().as_uri(), wait_until="load")

        page.pdf(
            path=str(pdf_path),
            format="A4",
            # print_background keeps the code blocks and callout boxes from rendering
            # as plain white, which is most of what makes the document readable.
            print_background=True,
            margin={"top": "14mm", "bottom": "16mm", "left": "12mm", "right": "12mm"},
            display_header_footer=True,
            header_template="<div></div>",
            footer_template=(
                '<div style="width:100%;font-size:8px;color:#94a3b8;'
                'padding:0 12mm;display:flex;justify-content:space-between;">'
                "<span>Nexaris AI Outreach Agent</span>"
                '<span class="pageNumber"></span>'
                "</div>"
            ),
        )
        browser.close()

    return pdf_path


def main() -> int:
    if len(sys.argv) < 2:
```

```
    print(__doc__)
    return 1

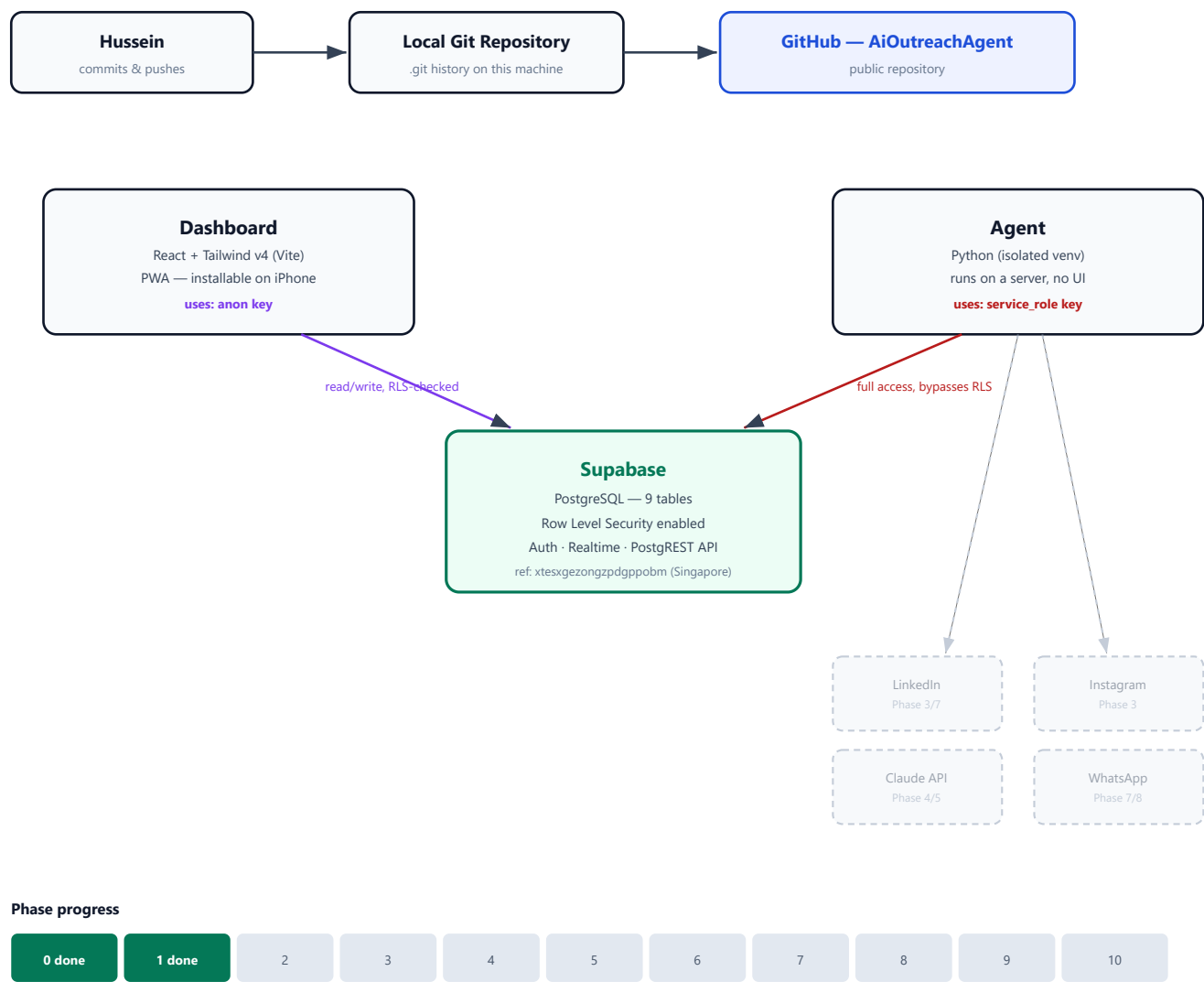
for arg in sys.argv[1:]:
    out = render(Path(arg))
    size_kb = out.stat().st_size / 1024
    print(f"wrote {out}  ({size_kb:.0f} KB)")

return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

C. How everything connects — the system as built after Phase 1

The diagram below shows the real, live shape of the system today: two independent applications, connected only through the shared database, plus the version-control flow that gets code from this machine onto GitHub. Grey boxes are not built yet — they show where future phases attach.



■ Complete & verified ■ Not started ■ Anon key + RLS path ■ Service-role key path (bypasses RLS)

In plain terms: the dashboard and the agent never talk to each other directly — the only thing connecting them is the Supabase database, reached two different ways with two different levels of trust. Version control (Git/GitHub) is a completely separate flow that moves code, not data. Every box on the right, in grey, is where the next phases attach.